MediChain

Hospital Management Platform with a
Zero-Trust Security Layer and a Tamper-Evident Ledger

Complete Project Documentation

Version 2: the completed system, explained module by module with the real source code

Repository	https://github.com/nikhil-k3312/hospital-management-system
Fork	https://github.com/VSshashank/hospital-management-system
Code state	branch `feature/complete-medichain`
Design source	MediChain Research Paper v1.0 (AY 2025–2026), `PLAN.md`, `AUDIT.md`
Team	AS (Blockchain & Security), NI (Backend & API), SI (Full-Stack Integration), SH (Frontend & UI/UX)
Prepared	October 2026

Who this is for

Anyone who has never seen the project: a new developer, a reviewer, an examiner. After reading it you should know what MediChain does and why, how every feature works from the browser down to the database, how to run and demonstrate it, and where to change it. Every code listing in this document is copied from the repository; file names are given so you can open the original.

Contents

1 Introduction **1**

1.1 What MediChain is . . . 1

1.2 The problem it addresses . . . 1

1.3 What changed from the starting point . . . 2

1.4 How to read this document . . . 2

2 Requirements **3**

2.1 Users and roles . . . 3

2.2 Functional requirements . . . 3

2.3 Non-functional requirements . . . 5

2.4 Constraints . . . 5

3 Architecture **6**

3.1 The big picture . . . 6

3.2 Inside the API . . . 7

3.3 Technology stack and why . . . 8

3.4 Architecture decisions . . . 8

3.5 Why a hash chain is enough . . . 9

3.6 Repository layout . . . 9

4 Setup and Running **10**

4.1 Prerequisites . . . 10

4.2 Environment variables . . . 10

4.3 Running locally . . . 11

4.4 Running with Docker . . . 11

4.5 Scripts . . . 12

4.6 The demo data . . . 12

5 Backend Foundation **13**

5.1 Life of a request . . . 13

5.2 Configuration: `src/config/config.js` . . . 13

5.3 Process start-up: `src/server.js` . . . 14

5.4 The Express application: `src/app.js` . . . 15

5.4.1 Security headers and the Content-Security-Policy . . . 15

5.4.2 Rate limits . . . 15

5.4.3 Routes and the bundled frontend . . . 16

5.5 Errors: `src/utils/errors.js` and the error handler . . . 16

5.6 Input validation: src/middleware/validate.js . . . 17
5.7 Databases: src/db/ . . . 18
5.8 Hashing helpers: src/utils/hash.js . . . 19

6 Authentication and Sessions 20
6.1 The User model: src/models/User.js . . . 20
6.2 Registration . . . 21
6.3 Two-step login . . . 22
6.3.1 Step 1: POST /api/auth/login/request-otp . . . 22
6.3.2 The OTP service: src/services/otpService.js . . . 24
6.3.3 Step 2: POST /api/auth/login/verify-otp . . . 24
6.4 Tokens: src/services/tokenService.js . . . 25
6.5 Refresh-token rotation . . . 26
6.6 Logout, password change and deactivation . . . 27
6.7 Redis key space . . . 28

7 The Zero-Trust Layer 29
7.1 authenticate . . . 29
7.1.1 Continuous verification with a session cache . . . 30
7.2 The trust score . . . 30
7.3 The policy engine . . . 31
7.4 Break-glass emergency access . . . 32
7.5 The audit trail: src/services/auditService.js . . . 33

8 The Tamper-Evident Ledger 34
8.1 The idea in one picture . . . 34
8.2 Two chains . . . 34
8.3 The entry: src/models/LedgerEntry.js . . . 34
8.4 Writing an entry: src/services/ledgerService.js . . . 35
8.4.1 Concurrency: two requests at once . . . 36
8.4.2 The shortcut used by every module . . . 36
8.5 Verifying . . . 37
8.5.1 Walking a chain . . . 37
8.5.2 Checking a document against the ledger . . . 38
8.5.3 The full audit: GET /api/ledger/verify . . . 38
8.6 Demonstrating tamper detection . . . 39
8.7 Limits and how to close them . . . 40

9 Appointments 41
9.1 Generating slots from availability . . . 41
9.2 Double-booking prevention . . . 42
9.3 Booking: POST /api/appointments . . . 42

9.4 Status changes, rescheduling and cancelling . . . 43

10 Medical Records, Consent and Files **45**
10.1 Consent model . . . 45
10.2 The consent gate: `src/services/consentService.js` . . . 46
10.3 Records: `src/modules/records/records.routes.js` . . . 47
10.3.1 Amending a record . . . 47
10.4 Files: GridFS with hashes in the ledger . . . 48
10.4.1 Downloading . . . 49
10.4.2 Verifying a record and its files . . . 49

11 Prescriptions, Vital Signs and IoT Devices **51**
11.1 Prescriptions . . . 51
11.2 Vital signs . . . 52
11.3 IoT devices . . . 53
11.3.1 Registration and keys . . . 53
11.3.2 Ingestion . . . 53
11.3.3 The doctor's live monitor . . . 53
11.3.4 The simulator . . . 54

12 Billing and Insurance Claims **55**
12.1 Bills . . . 55
12.2 Claims . . . 56
12.2.1 Filing a claim . . . 56
12.2.2 The claim state machine . . . 56
12.2.3 Settlements and reports . . . 58

13 Administration, Notifications and Settings **59**
13.1 Credential verification . . . 59
13.2 User management . . . 59
13.3 Live statistics . . . 60
13.4 Hospital settings . . . 60
13.5 Access audit . . . 60
13.6 IoT device registry . . . 60
13.7 Notifications . . . 60

14 Frontend **62**
14.1 Structure . . . 62
14.2 The API client: `js/api.js` . . . 62
14.3 The dashboard shell: `js/portal.js` . . . 64
14.3.1 Start-up . . . 64
14.3.2 Rendering a page: real loading, empty and error states . . . 64

14.3.3 Escaping: the second line of XSS defence . . . 65
14.4 A complete page: the patient's Permissions screen . . . 65
14.5 Login: `loginscript.js` . . . 66
14.6 What each dashboard shows . . . 67

15 Reference: Data Model and API **69**
15.1 MongoDB collections . . . 69
15.2 Response format and error codes . . . 69
15.3 Endpoints . . . 70
15.4 Glossary . . . 73

16 Testing and Deployment **74**
16.1 Backend tests . . . 74
16.1.1 Isolation . . . 74
16.1.2 Helpers that use the real login flow . . . 74
16.1.3 What is tested . . . 75
16.2 Frontend tests . . . 75
16.3 Docker . . . 76
16.4 Continuous integration . . . 76
16.5 Production checklist . . . 76

17 Demo, Security Summary and Future Work **77**
17.1 Demo script . . . 77
17.2 Security summary . . . 77
17.3 Known limitations and future work . . . 78
17.4 Verification record . . . 79

List of Figures

3.1 System context 6
3.2 Layers inside the API 7

6.1 Password, then one-time code 22

8.1 Three entries of the records chain 34

9.1 Appointment states 43

10.1 Consent lifecycle 45

12.1 Bill states 55
12.2 Claim states 57

List of Tables

1.1 Starting point versus completed system 2

2.1 Roles 3
2.2 Functional requirements 3
2.3 Non-functional requirements 5

3.1 Technology choices 8

4.1 Environment variables 10
4.2 npm scripts in `backend/services/api` 12

6.1 The two tokens 25
6.2 What lives in Redis 28

7.1 Trust score signals 31

13.1 Who is notified of what 61

14.1 Frontend files 62
14.2 Dashboard pages and their endpoints 67

15.1 Collections 69
15.2 HTTP status codes 70
15.3 API endpoints 70

16.1 Backend test suites 75

17.1 Threats and controls 78

List of Code Listings

3.1 Repository layout . . . 9
4.1 First run on macOS . . . 11
4.2 Docker Compose . . . 11
4.3 How the seed derives demo emails (scripts/seed.js) . . . 12
5.1 Start-up validation (src/config/config.js) . . . 13
5.2 Start-up order (src/server.js) . . . 14
5.3 helmet and CSP (src/app.js) . . . 15
5.4 Two rate limiters (src/app.js) . . . 15
5.5 Module mounting and static frontend (src/app.js) . . . 16
5.6 AppError and asyncHandler (src/utils/errors.js) . . . 16
5.7 Central error handler, mapping section (src/middleware/errorHandler.js) . . . 17
5.8 The validate middleware (src/middleware/validate.js) . . . 17
5.9 GridFS bucket (src/db/mongo.js) . . . 18
5.10 Redis connection that fails loudly (src/db/redis.js) . . . 18
5.11 Canonical JSON and hashing (src/utils/hash.js) . . . 19
6.1 User schema, key fields (src/models/User.js) . . . 20
6.2 Password hashing and ledger address (src/models/User.js) . . . 20
6.3 Never leaking secrets in JSON (src/models/User.js) . . . 21
6.4 register, core (src/modules/auth/auth.controller.js) . . . 21
6.5 requestOtp (src/modules/auth/auth.controller.js) . . . 22
6.6 Lockout bookkeeping (src/modules/auth/auth.controller.js) . . . 23
6.7 Generating, storing and checking codes (src/services/otpService.js) . . . 24
6.8 verifyOtp, success path (src/modules/auth/auth.controller.js) . . . 24
6.9 Issuing tokens (src/services/tokenService.js) . . . 25
6.10 The refresh cookie (src/modules/auth/auth.controller.js) . . . 26
6.11 refreshToken (src/modules/auth/auth.controller.js) . . . 26
6.12 Revocation on logout (src/modules/auth/auth.controller.js) and blacklisting (tokenService.js) . . . 27
7.1 authenticate (src/middleware/auth.js) . . . 29
7.2 loadSessionState (src/middleware/auth.js) . . . 30
7.3 calculateTrustScore (src/services/trustService.js) . . . 30
7.4 evaluatePolicy (src/services/trustService.js) . . . 31
7.5 recordAccessDecision (src/services/auditService.js) . . . 33
8.1 LedgerEntry schema (src/models/LedgerEntry.js) . . . 35
8.2 computeEntryHash and appendOnce (src/services/ledgerService.js) . . . 35
8.3 Serialised appends with retry (src/services/ledgerService.js) . . . 36
8.4 recordEvent (src/services/ledgerService.js) . . . 37

8.5 What the ledger protects for a record (src/models/MedicalRecord.js) . . . 37
8.6 verifyChain (src/services/ledgerService.js) . . . 37
8.7 verifyEntity (src/services/ledgerService.js) . . . 38
8.8 Full integrity audit (src/modules/ledger/ledger.routes.js) . . . 38
8.9 The insider edit (scripts/tamper-demo.js) . . . 39
8.10 Running the demonstration . . . 39
8.11 Tamper detection test (tests/integration/records.test.js) . . . 40
9.1 buildDaySlots and isValidSlot (src/modules/appointments/slots.js) . . . 41
9.2 Partial unique index (src/models/Appointment.js) . . . 42
9.3 Booking a slot (src/modules/appointments/appointments.routes.js) . . . 42
9.4 Allowed doctor transitions (appointments.routes.js) . . . 43
10.1 AccessGrant (src/models/AccessGrant.js) . . . 45
10.2 A patient grants access directly (src/modules/consents/consents.routes.js) . . . 46
10.3 assertCanAccessPatient (src/services/consentService.js) . . . 46
10.4 Record policies and creation (records.routes.js) . . . 47
10.5 Versioned amendment (records.routes.js) . . . 48
10.6 Upload limits (records.routes.js) . . . 48
10.7 Storing a file and anchoring its hash (records.routes.js) . . . 48
10.8 Consent-gated download (src/modules/records/files.routes.js) . . . 49
10.9 Per-record integrity check (records.routes.js) . . . 49
11.1 Refill request rules (prescriptions.routes.js) . . . 51
11.2 The doctor decides (prescriptions.routes.js) . . . 51
11.3 Normal ranges and the abnormal check (src/models/VitalReading.js) . . . 52
11.4 Saving a reading and raising alerts (vitals.routes.js) . . . 52
11.5 Device model (src/models/Device.js) . . . 53
11.6 POST /api/devices/readings (vitals.routes.js) . . . 53
11.7 Simulated readings (scripts/iot-simulator.js) . . . 54
12.1 createBillForAppointment (src/modules/billing/billing.service.js) . . . 55
12.2 POST /api/claims (claims.routes.js) . . . 56
12.3 Allowed transitions (src/models/Claim.js) . . . 56
12.4 Processing a claim (claims.routes.js) . . . 57
13.1 Approving or rejecting (admin.routes.js) . . . 59
13.2 Parallel aggregation (admin.routes.js, abridged) . . . 60
13.3 notify and notifyRole (src/services/notificationService.js) . . . 61
14.1 Script tags of frontend/patient.html . . . 62
14.2 request with transparent refresh (frontend/js/api.js) . . . 63
14.3 One shared refresh (frontend/js/api.js) . . . 63
14.4 Portal.start (frontend/js/portal.js) . . . 64
14.5 showPage (frontend/js/portal.js) . . . 64
14.6 The error state (frontend/js/ui.js) . . . 65

14.7 esc (`frontend/js/ui.js`) 65
14.8 Data loading and handlers (`frontend/js/patient.js`, abridged) 65
14.9 Two-step sign-in (`frontend/loginscript.js`, abridged) 66
16.1 Test environment (`tests/env.js`) 74
16.2 Logging in inside a test (`tests/helpers.js`) 74
16.3 The double-booking race (`tests/integration/appointments.test.js`) 75
16.4 `Dockerfile` 76

CHAPTER 1

Introduction

1.1 What MediChain is

MediChain is a web platform that runs the day-to-day information flow of a hospital for four kinds of people:

Patients book appointments, read their own medical records, decide which doctors may see those records and for how long, follow their medications and home-monitoring vital signs, pay bills and file insurance claims.

Doctors manage their schedule, ask patients for consent, write and amend medical records, attach files such as lab reports, prescribe, watch live vital signs from patients' devices, and in a genuine emergency open a record without consent ("break-glass"), which is logged and reported.

Administrators verify the credentials of new doctors and insurance agents, manage user accounts and IoT devices, configure the hospital, read the security audit trail and check the integrity of the whole database.

Insurance agents work through a queue of claims, approve or deny them, record settlements and read financial reports.

Two ideas run through the whole system and make it more than a CRUD application:

1. **Zero trust.** No request is trusted because of where it comes from or because the user logged in earlier. Every API call is authenticated, scored, checked against the user's role, approval status and (for medical data) the patient's consent, and the decision is recorded.
2. **Tamper evidence.** Every change to clinical or financial data, and every access decision, is written to a hash chain: a sequence of entries where each one contains the SHA-256 hash of the previous one. If anyone edits the database directly, bypassing the application, the system can prove it.

1.2 The problem it addresses

Hospital data has three properties that make ordinary web-application practice insufficient:

- **It is extremely sensitive.** A diagnosis or a prescription must be visible only to the patient and the clinicians the patient has chosen. "Logged-in user with the doctor role" is not a good enough rule.
- **It must be trustworthy.** Insurance decisions, legal disputes and clinical care all depend on records being exactly what the doctor wrote. A silently edited record is worse than a missing one.
- **Emergencies happen.** An unconscious patient cannot grant consent. The system must allow emergency access without making it invisible.

The original MediChain research paper proposed Hyperledger Fabric (a blockchain) and IPFS (distributed file storage) to meet these needs. This implementation keeps the guarantees and replaces the heavy infrastructure with a hash-chain ledger and file hashes stored in MongoDB (Chapter 3 explains the reasoning).

1.3 What changed from the starting point

When this work began, the repository contained a polished but static frontend (every number on every dashboard was hardcoded in JavaScript) and one backend service that handled login only. Table 1.1 summarises what exists now.

Table 1.1: Starting point versus completed system

Area	Before	Now
Backend	One auth service, 6 endpoints, a broken logout	One API service, 68 endpoints in 13 route files
Frontend data	Hardcoded arrays, simulated login	Every screen loads from the API; real loading, empty and error states
Login	Simulated in the browser	Password + emailed OTP, JWT with rotating refresh cookie
Appointments, records, claims, admin	Not implemented	Implemented, tested, wired to all four dashboards
Ledger	A JSON-lines audit file	Two hash chains in MongoDB, full verification, tamper demo
Files	None	GridFS storage, SHA-256 anchored in the ledger, consent-gated downloads
IoT vitals	Static numbers	Device API keys, ingestion endpoint, live monitor, alerts, simulator
Tests	None	105 backend + 20 frontend tests, about 93% backend line coverage
Deployment	None	Dockerfile, docker-compose, GitHub Actions CI

1.4 How to read this document

- Chapters 2–4 are the overview: requirements, architecture and how to run the project. Read these first.
- Chapters 5–13 walk through the backend one module at a time. Each section states *what* the module is for, *why* it is built that way, and then shows the real code with an explanation of each part.
- Chapter 14 does the same for the browser code.
- Chapters 15–17 are reference material: data model, full API list, tests, deployment, the demo script, the security model and what is left to do.

Conventions: file paths are relative to the repository root unless they start with `src/`, which means `backend/services/api/src`. Code listings are exact copies; where a listing omits lines, the gap is marked with a `// ...` comment.

CHAPTER 2

Requirements

This chapter lists what the system must do, per user role, and the quality bars it must meet. The "Where" column points to the chapter that explains the implementation. Every requirement listed here is implemented and covered by tests unless marked otherwise.

2.1 Users and roles

Table 2.1: Roles

Role	Who	How the account is created
`patient`	A person receiving care	Self-registration; active immediately
`doctor`	A clinician	Self-registration with licence number; *pending* until an admin verifies it, or created by an admin
`insurance_agent`	Claims officer of one insurer	Self-registration with company; *pending* until verified, or created by an admin
`admin`	Hospital IT / administration	Only by the seed script or another admin, never by self-registration

2.2 Functional requirements

Table 2.2: Functional requirements

ID	Requirement	Where
Accounts and login		
FR-1	Patients, doctors and agents can register; doctors and agents start as pending.	Ch. 6
FR-2	Login takes two factors: password, then a 6-digit code emailed to the user.	Ch. 6
FR-3	Sessions survive a page reload without storing a token in the browser's storage.	Ch. 6, 14
FR-4	Logout, password change and account deactivation end sessions immediately.	Ch. 6
FR-5	Repeated failures lock the account temporarily.	Ch. 6
Appointments		

ID	Requirement	Where
FR-6	Patients see doctors' free slots and book one; a slot can never be booked twice.	Ch. 9
FR-7	Doctors confirm, complete or mark no-show; patients reschedule; both can cancel.	Ch. 9
FR-8	Completing an appointment raises a bill automatically.	Ch. 12
Medical records and consent		
FR-9	A doctor can read or write a patient's records only with that patient's active, unexpired consent.	Ch. 10
FR-10	Doctors request access; patients approve, deny, grant directly and revoke.	Ch. 10
FR-11	Doctors and admins can use audited emergency (break-glass) access; the patient and admins are notified.	Ch. 7, 10
FR-12	Records keep a version history; every change is in the ledger.	Ch. 10
FR-13	Files (PDF, images, text) can be attached; their SHA-256 is recorded; downloads are consent-gated.	Ch. 10
Prescriptions and vitals		
FR-14	Doctors prescribe; patients request refills; doctors approve or deny.	Ch. 11
FR-15	Registered IoT devices send readings with an API key; patients can add readings manually.	Ch. 11
FR-16	Out-of-range readings alert the patient and every doctor with consent.	Ch. 11
Billing and insurance		
FR-17	Patients pay bills or file a claim against their policy.	Ch. 12
FR-18	Agents move claims through submitted → under review → approved/denied → settled, nothing else.	Ch. 12
FR-19	Agents see settlements and financial reports for their insurer.	Ch. 12
Administration and integrity		
FR-20	Admins verify or reject doctors and agents, activate/deactivate users, create staff accounts.	Ch. 13
FR-21	Admins register IoT devices and see live system statistics.	Ch. 13
FR-22	Admins can verify the entire ledger and see which documents were tampered with.	Ch. 8
FR-23	Every user gets in-app notifications for events that concern them.	Ch. 13

2.3 Non-functional requirements

Table 2.3: Non-functional requirements

Quality	Requirement and how it is met	Status
No fake data	Application code never fabricates data; only the seed script inserts demo data.	**Done**
Fail closed	If Redis (OTP, sessions, revocation) is down, protected requests get 503, never a free pass.	**Done**
Fail fast	The server refuses to start with missing or unsafe configuration.	**Done**
Input validation	Every route validates params, query and body with Joi.	**Done**
XSS defence	Strict Content-Security-Policy; all dynamic text escaped; no inline scripts.	**Done**
Test coverage	Backend and frontend test suites; target 90% backend lines.	**Done** (93%)
Performance	API under 200 ms, pages under 2 s (paper §6.3).	**Partial** (not load-tested)
Security scan	OWASP ZAP pass against a running instance.	**Not built**

2.4 Constraints

- Single team, academic timeline: infrastructure must be runnable on a laptop (MongoDB, Redis, Node.js) or with one `docker compose` command.
- The existing frontend design (HTML/CSS) is kept; it is wired to the API rather than rewritten in another framework (decision D4, Section 3.4).
- All configuration and secrets come from environment variables.

CHAPTER 3

Architecture

3.1 The big picture

MediChain is a classic three-tier web application with one important addition: the ledger, which lives in the same database but is written only by one service and is never updated or deleted.

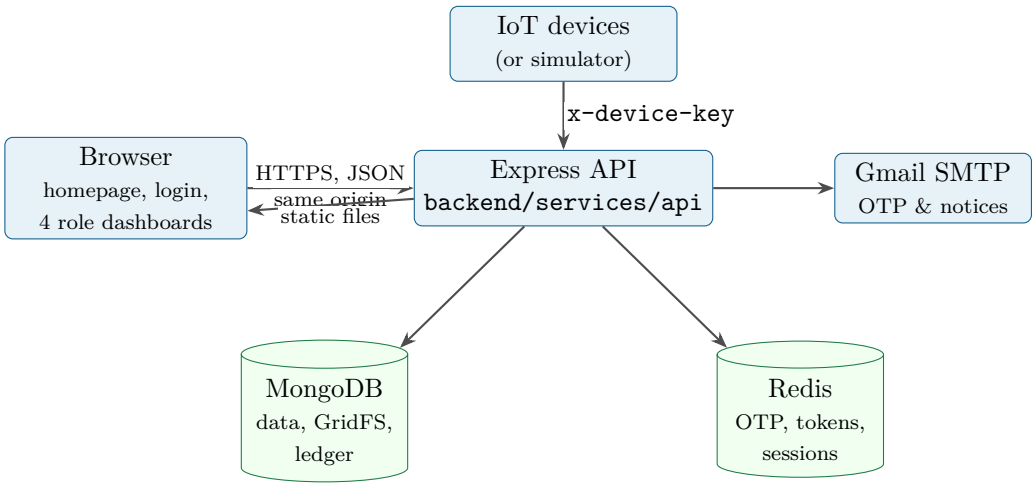


Figure 3.1: System context

- **Browser.** Plain HTML, CSS and JavaScript, served by the API itself. Being on the same origin lets the refresh-token cookie be `SameSite=Strict` and removes the need for CORS in the browser.
- **API.** One Node.js/Express service. It is split internally into modules (auth, appointments, records, . . .) that share middleware and services.
- **MongoDB** stores all business data, uploaded files (GridFS) and the ledger (`ledgerentries` collection).
- **Redis** stores short-lived security state: hashed OTPs, refresh token ids, the access-token blacklist, cached session state and attempt counters. Everything in Redis expires on its own.
- **SMTP** (Gmail) delivers OTP codes, welcome and verification emails.
- **IoT devices** post readings with their own API key; they never hold a user session.

3.2 Inside the API

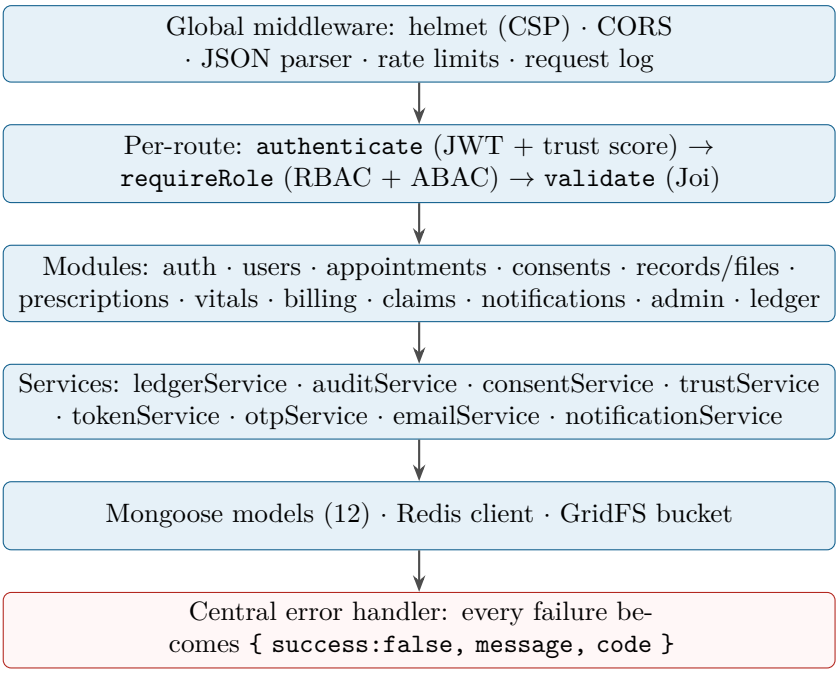


Figure 3.2: Layers inside the API

The rule that keeps this maintainable: *route handlers do HTTP; services do policy.* For example, every module that touches a patient's data calls the same `consentService.assertCanAccessPatient`, and every module that changes protected data calls the same `ledgerService.recordEvent`. There is exactly one implementation of "may this doctor see this patient?" and one of "write a ledger entry".

3.3 Technology stack and why

Table 3.1: Technology choices

Concern	Choice	Why
Runtime / framework	Node.js 18+, Express 4	Already used by the original auth service; small and well understood.
Database	MongoDB + Mongoose 8	Already in the repo. Flexible documents suit per-role profiles; unique and partial indexes give real integrity guarantees.
Files	GridFS	Lives in the same MongoDB, no extra service; streams large files.
Security state	Redis 4 client	TTL on every key: OTPs and blacklist entries clean themselves up.
Auth	bcryptjs, jsonwebtoken	Standard password hashing and signed tokens.
Validation	Joi 17	Declarative schemas, all errors reported at once.
HTTP hardening	helmet, express-rate-limit, cors	CSP and security headers, brute-force limits.
Uploads	multer 2	Multipart parsing with type and size limits.
Email	Nodemailer 6	SMTP (Gmail) in production; console and in-memory transports for development and tests.
Logging	winston 3	Console plus rotating files in `backend/logs` (git-ignored).
Frontend	Vanilla JS, Chart.js 4.4.1, Lucide 0.460.0	No build step; the existing pages were kept (D4).
Tests	Jest 29, Supertest 7, jsdom, Testing Library	Real HTTP requests against real Mongo and Redis.

3.4 Architecture decisions

`PLAN.md` §3 locked six decisions before any feature code was written. Their outcomes:

D1 – One database.
MongoDB for everything; no PostgreSQL. A second database would add risk with no user-facing benefit at this scale.

D2 – Hash chain instead of Hyperledger Fabric.
Fabric needs a network of certificate authorities, peers and an orderer, plus chaincode lifecycle management: a project in itself. The paper's actual requirement is *immutable, tamper-evident and auditable*. A hash chain meets that inside the existing database (Chapter 8 shows how).

D3 – GridFS instead of IPFS.
The pattern from the paper is kept: the file lives in storage, its SHA-256 lives in the ledger. Only the storage backend differs.

D4 – Keep the vanilla frontend.
The existing HTML/CSS was substantial, so it was wired to the API instead of rewritten in React. Shared state that Redux would have held lives in two small modules (`js/api.js`, `js/portal.js`).

D5 – Redis has a defined job.
OTPs, refresh tokens, revocation, session cache and attempt counters. Nothing else.

D6 – Trust score is a real feature.
The `MIN_TRUST_SCORE` and `HIGH_SENSITIVITY_TRUST_SCORE` settings were found to belong to a documented zero-trust design. They are now enforced on every request (Chapter 7).

3.5 Why a hash chain is enough

A blockchain such as Fabric solves two problems: *tamper evidence* (you can tell if history was changed) and *decentralised trust* (no single party can change history, because many independent parties hold copies). MediChain is run by one hospital, so the second problem does not arise in the same form: there is no group of mutually distrusting organisations. The first problem is exactly what a hash chain solves. Section 8.7 discusses the remaining gap (an attacker who can rewrite the *whole* chain) and how to close it by publishing the chain's head hash externally.

3.6 Repository layout

Listing 3.1: Repository layout

```
backend/services/api/
  src/
    app.js, server.js        Express app and process start-up
    config/config.js          environment → config object, validation
    db/                       mongo.js (connection, GridFS), redis.js
    middleware/               auth.js, validate.js, errorHandler.js
    models/                   12 Mongoose models
    services/                 ledger, audit, consent, trust, token, otp, email, notification
    modules/<area>/           route files (and helpers) per feature area
    utils/                    errors.js, hash.js, logger.js
  scripts/                    seed, IoT simulator, tamper demo, ledger verifier
  tests/                      unit/ and integration/ (Jest + Supertest)
frontend/
  homepage.html, login.html   public pages
  patient.html, doctor.html, admin.html, insurance.html
  css/portal.css              shared dashboard styles
  js/api.js, ui.js, portal.js shared client code
  js/patient.js, doctor.js, admin.js, insurance.js
  tests/                      Jest + jsdom tests
docs/                         this document, DEMO.md, zero-trust notes
Dockerfile, docker-compose.yml, .github/workflows/ci.yml
```

CHAPTER 4

Setup and Running

4.1 Prerequisites

- Node.js 18 or newer (developed on 24, Docker image uses 20).
- MongoDB 6+ and Redis 6+, either installed locally (macOS: `brew install mongodb-community redis`) or through Docker.
- For real OTP emails: a Gmail account with 2-step verification and an *app password* (Google Account → Security → App passwords).

4.2 Environment variables

All configuration comes from `backend/services/api/.env` (copied from `.env.example` at the repository root; the real `.env` is git-ignored).

Table 4.1: Environment variables

Variable	Default	Meaning
`PORT`	8001	HTTP port (API and frontend)
`NODE_ENV`	development	`production` enables secure cookies, hides error details, requires SMTP
`MONGODB_URI`	*required*	e.g. `mongodb://127.0.0.1:27017/medichain`
`JWT_SECRET`	*required*	Signs access tokens; `openssl rand -hex 64`
`JWT_REFRESH_SECRET`	*required*	Signs refresh tokens; must differ from `JWT_SECRET`
`JWT_EXPIRY`	15m	Access-token lifetime
`JWT_REFRESH_EXPIRY_DAYS`	7	Refresh-token lifetime
`EMAIL_TRANSPORT`	smtp	`smtp` sends mail; `console` prints it to the log (development only); `json` keeps it in memory (tests)
`EMAIL_SERVICE`	–	Nodemailer service name, e.g. `gmail`
`EMAIL_USER`, `EMAIL_PASSWORD`	*required for smtp*	Gmail address and app password
`EMAIL_FROM`	`EMAIL_USER`	Sender shown in emails
`OTP_EXPIRY_MINUTES`	10	OTP lifetime
`OTP_LENGTH`	6	Digits per OTP
`OTP_MAX_ATTEMPTS`	10	OTP requests + wrong codes per email per 15 minutes

Variable	Default	Meaning
REDIS_HOST, REDIS_PORT, REDIS_DB	localhost, 6379, 0	Redis connection
MIN_TRUST_SCORE	50	Trust score needed for any authenticated request
HIGH_SENSITIVITY_TRUST_SCORE	70	Trust score needed for medical records and files
MAX_FAILED_LOGINS	5	Failures before the account locks
ACCOUNT_LOCK_MINUTES	30	Lock duration
RATE_LIMIT_WINDOW_MINUTES	15	Rate-limit window
RATE_LIMIT_MAX	1000	Requests per IP per window, all of /api
AUTH_RATE_LIMIT_MAX	30	Requests per IP per window to login and register
MAX_UPLOAD_MB	10	Largest accepted file
ALLOWED_ORIGINS	http://localhost:8001	Cross-origin callers (the bundled frontend needs none)
SEED_EMAIL_BASE	–	Seed only: demo accounts become plus-addresses of this email
SEED_DEMO_PASSWORD	–	Seed only: password shared by all demo accounts

Plus-addressing for the demo

Gmail delivers you+patient@gmail.com to you@gmail.com. The seed script creates every demo account as such an address of SEED_EMAIL_BASE, so one inbox receives the OTP codes of all eight demo users, yet each account still has its own unique email.

4.3 Running locally

Listing 4.1: First run on macOS

```
brew services start mongodb-community
brew services start redis
cp .env.example backend/services/api/.env     # then edit it (Section 4.2)
cd backend/services/api
npm install
npm run seed -- --reset                        # demo hospital
npm run dev                                    # http://localhost:8001
```

4.4 Running with Docker

Listing 4.2: Docker Compose

```
docker compose up --build -d                   # mongo + redis + api
docker compose exec api node scripts/seed.js --reset
```

The compose file runs the API with `NODE_ENV=production`, which requires `EMAIL_TRANSPORT=smtp`. To try it with console OTPs instead, start it with `NODE_ENV=development docker compose up -build -d`.

4.5 Scripts

Table 4.2: npm scripts in `backend/services/api`

Command	Purpose
`npm run dev / npm start`	Start with auto-reload (nodemon) / plain node
`npm test`	All backend tests (own databases, never touches dev data)
`npm run test:coverage`	Tests with a coverage report
`npm run seed - -reset`	Drop the database and load the demo hospital
`npm run demo:iot`	Stream device readings (`-interval N`, `-spike`)
`npm run demo:tamper`	Edit a record directly in MongoDB (`-ledger`, `-restore`)
`npm run ledger:verify`	Full integrity check from the command line

4.6 The demo data

`scripts/seed.js` creates a small but realistic hospital so that every screen has something to show:

- hospital settings (departments, four insurers, booking window);
- one admin; Dr Ananya Iyer (Cardiology) and Dr Rahul Menon (Endocrinology), both verified; Dr Kavya Reddy (Dermatology), pending; agent Priya Sharma (Star Health);
- patients Arjun Mehta (hypertension, consent to Dr Iyer, BP monitor), Meera Nair (type 2 diabetes, pulse oximeter) and Rohan Gupta (no consent to anyone: the break-glass patient);
- past, today's and upcoming appointments; records with real attached files; prescriptions including a pending refill; bills and claims in several states; seven days of device readings; notifications.

The seed is a script, not application code: it writes directly to MongoDB. But it writes every protected document through `ledgerService` exactly as the API would, so "Verify ledger" reports a valid chain straight after seeding.

Listing 4.3: How the seed derives demo emails (`scripts/seed.js`)

```
const base = requireEnv('SEED_EMAIL_BASE');
// ...
const [local, domain] = base.split('@');
const email = (tag) => `${local}+${tag}@${domain}`;   // you+patient@gmail.com
```

CHAPTER 5

Backend Foundation

Before any feature, every request passes through the same foundation: the configuration, the Express application with its security middleware, input validation and the central error handler. This chapter explains each piece.

5.1 Life of a request

Take `GET /api/records?patientId=...` sent by a doctor. In order:

1. **helmet** adds security headers (CSP, `nosniff`, ...).
2. **CORS**, **cookie-parser** and the **JSON parser** (100 kB limit) run.
3. The **rate limiter** counts the request against the caller's IP.
4. The router for `/api/records` runs **authenticate**: verify the JWT, check it is not revoked, load the account state, compute the trust score (Chapter 7).
5. **requireRole**`('patient','doctor', {minTrustScore:70, allowBreakGlass:true})` checks the role, approval status and trust score.
6. **validate** checks the query string against a Joi schema.
7. The handler calls **consentService** (does this doctor have consent?) and then MongoDB.
8. If anything throws, the **error handler** turns it into JSON.
9. Allow and deny decisions are written to the **access chain** of the ledger.

5.2 Configuration: `src/config/config.js`

Purpose. Turn environment variables into one typed object, and refuse to start if the configuration is missing or unsafe.

Why. The original service started with missing secrets and failed later on the first login, with an obscure error. Failing at start-up is cheaper to debug and can never half-work in production.

Listing 5.1: Start-up validation (`src/config/config.js`)

```
function validateConfig(cfg = config, rawEnv = env) {
  const required = ['MONGODB_URI', 'JWT_SECRET', 'JWT_REFRESH_SECRET'];
  if (cfg.email.transport === 'smtp') {
    required.push('EMAIL_USER', 'EMAIL_PASSWORD');
  }

  const missing = required.filter((key) => !rawEnv[key] || String(rawEnv[key]).trim() === '');
  if (missing.length > 0) {
    throw new Error(`Missing required environment variables: ${missing.join(', ')}`);
  }

  if (!['smtp', 'console', 'json'].includes(cfg.email.transport)) {
    throw new Error(`EMAIL_TRANSPORT must be smtp, console or json (got "${cfg.email.transport}")
    `);
```

```
  }

  // Printing OTPs to a log is only acceptable on a developer machine.
  if (cfg.isProduction && cfg.email.transport !== 'smtp') {
    throw new Error('EMAIL_TRANSPORT must be "smtp" in production');
  }

  if (rawEnv.JWT_SECRET === rawEnv.JWT_REFRESH_SECRET) {
    throw new Error('JWT_SECRET and JWT_REFRESH_SECRET must be different');
  }

  return true;
}
```

How it works

- Email credentials are required only when mail is really sent (`smtp`).
- The `console` transport prints OTP codes to the log, which is convenient on a laptop and a security hole on a server, so production rejects it.
- If both JWT secrets were equal, a refresh token could be presented as an access token. Requiring different secrets closes that.
- The function takes `cfg` and `rawEnv` as parameters so unit tests can call it with fake environments.

5.3 Process start-up: `src/server.js`

Listing 5.2: Start-up order (`src/server.js`)

```
async function start() {
  try {
    config.validateConfig();
    await connectMongo();
    await connectRedis();
    await HospitalSettings.getSingleton();
    await emailService.verifyConnection();

    const app = require('./app');
    const server = app.listen(config.port, () => {
      logger.info(`MediChain API listening on http://localhost:${config.port} (${config.nodeEnv})
    `);
    });

    const shutdown = (signal) => {
      logger.info(`${signal} received, shutting down`);
      server.close(async () => {
        await Promise.allSettled([disconnectMongo(), disconnectRedis()]);
        process.exit(0);
      });
      setTimeout(() => process.exit(1), 10000).unref();
    };
    process.on('SIGTERM', () => shutdown('SIGTERM'));
    process.on('SIGINT', () => shutdown('SIGINT'));
  } catch (error) {
    logger.error(`Failed to start: ${error.message}`);
    process.exit(1);
  }
}
```

How it works

The server listens only after every dependency answered. `connectRedis` sends a `PING` and throws if Redis is not there (the old service swallowed this error and then failed every login). `HospitalSettings.getSingleton()` creates the settings document on first run. `verifyConnection` logs into SMTP so a wrong Gmail app password shows up in the log at start-up, not at the first login. On `SIGTERM` (Docker stop) the server stops accepting connections, finishes in-flight requests and closes the databases; if that takes longer than 10 s it exits anyway.

5.4 The Express application: `src/app.js`

5.4.1 Security headers and the Content-Security-Policy

Listing 5.3: helmet and CSP (`src/app.js`)

```
// The frontend is served by this same server, so a strict Content-Security-Policy
// can allow scripts only from this origin plus two pinned CDNs. Inline <script>
// and onclick="" handlers are blocked, which shuts the main door to XSS.
app.use(helmet({
  contentSecurityPolicy: {
    useDefaults: true,
    directives: {
      'script-src': ["'self'", 'https://cdn.jsdelivr.net', 'https://unpkg.com'],
      'style-src': ["'self'", "'unsafe-inline'", 'https://fonts.googleapis.com', 'https://cdn.
    jsdelivr.net'],
      'font-src': ["'self'", 'https://fonts.gstatic.com', 'data:'],
      'img-src': ["'self'", 'data:', 'blob:', 'https://images.pexels.com'],
      'connect-src': ["'self'"],
      'frame-src': ["'self'", 'blob:'],
      'object-src': ["'self'", 'blob:'],
      'upgrade-insecure-requests': config.isProduction ? [] : null,
    },
  },
  crossOriginEmbedderPolicy: false,
}));
```

How it works

A Content-Security-Policy tells the browser which sources it may load code from. With `script-src 'self'` and no `'unsafe-inline'`, even if an attacker manages to get `<script>alert(1)</script>` into a patient's name, the browser refuses to run it. This is why the frontend has no inline scripts and no `onclick=` attributes: all handlers are attached from `.js` files. `connect-src 'self'` means page scripts can only call this API. `blob:` in `frame-src`/`object-src` lets the dashboards preview downloaded PDFs.

5.4.2 Rate limits

Listing 5.4: Two rate limiters (`src/app.js`)

```
app.use('/api/', rateLimit({
  windowMs, max: config.rateLimit.max, standardHeaders: true, legacyHeaders: false,
  message: limiterResponse('Too many requests from this IP, please try again later.'),
  skip: () => config.isTest,
}));
```

```
// Stricter budget for the endpoints an attacker would hammer to guess passwords or codes.
const authLimiter = rateLimit({
  windowMs, max: config.rateLimit.authMax, standardHeaders: true, legacyHeaders: false,
  message: limiterResponse('Too many login attempts from this IP. Try again in 15 minutes.'),
  skip: () => config.isTest,
});
app.use(['/api/auth/login', '/api/auth/register'], authLimiter);
```

The general limit protects the whole API; the second, much smaller budget (30 per 15 minutes by default) applies to login and registration, the endpoints used for password and OTP guessing. Per-account limits (lockout, OTP attempts) are added on top in Chapter 6, so an attacker spreading requests over many IPs still hits a wall.

5.4.3 Routes and the bundled frontend

Listing 5.5: Module mounting and static frontend (src/app.js)

```
app.use('/api/auth', require('./modules/auth/auth.routes'));
app.use('/api', require('./modules/users/users.routes'));
app.use('/api/appointments', require('./modules/appointments/appointments.routes'));
app.use('/api/consents', require('./modules/consents/consents.routes'));
app.use('/api/records', require('./modules/records/records.routes'));
app.use('/api/files', require('./modules/records/files.routes'));
app.use('/api/prescriptions', require('./modules/prescriptions/prescriptions.routes'));
app.use('/api', require('./modules/vitals/vitals.routes'));
app.use('/api/bills', require('./modules/billing/billing.routes'));
app.use('/api/claims', require('./modules/claims/claims.routes'));
app.use('/api/notifications', require('./modules/notifications/notifications.routes'));
app.use('/api/admin', require('./modules/admin/admin.routes'));
app.use('/api/ledger', require('./modules/ledger/ledger.routes'));
app.use('/api', notFoundHandler);

// Same origin as the API: the SameSite=Strict refresh cookie just works and no
// CORS is needed in the browser. Development files are never served.
app.use((req, res, next) => (/^\/(node_modules|tests|package(-lock)?\.json|jest\.config)/.test(
    req.path) ? notFoundHandler(req, res) : next()));
app.use(express.static(config.frontendDir, { index: 'homepage.html', extensions: ['html'] }));
```

The `/api` 404 handler is mounted *before* the static files, so a mistyped API path returns a JSON 404 instead of an HTML page. The regular expression blocks the frontend's development files (its `node_modules`, tests and `package.json`) from being served.

5.5 Errors: src/utils/errors.js and the error handler

Listing 5.6: AppError and asyncHandler (src/utils/errors.js)

```
class AppError extends Error {
  constructor(status, message, code, details) {
    super(message);
    this.name = 'AppError';
    this.status = status;
    this.code = code || undefined;
    this.details = details || undefined;
  }
}

const badRequest = (message, details) => new AppError(400, message, 'BAD_REQUEST', details);
const unauthorized = (message, code = 'UNAUTHORIZED') => new AppError(401, message, code);
const forbidden = (message, code = 'FORBIDDEN') => new AppError(403, message, code);
```

```
const notFound = (message = 'Resource not found') => new AppError(404, message, 'NOT_FOUND');
const conflict = (message, code = 'CONFLICT') => new AppError(409, message, code);
// ...
const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
```

How it works

Handlers simply `throw conflict('This slot has just been booked...', 'SLOT_TAKEN')`. Express 4 does not catch errors thrown from `async` functions, so every handler is wrapped in `asyncHandler`, which forwards a rejected promise to `next(error)`. The central handler then produces the response. The machine-readable `code` lets the frontend react to specific errors (for example `SESSION_ENDED` sends the user back to the login page).

Listing 5.7: Central error handler, mapping section (src/middleware/errorHandler.js)

```
  if (err.message === 'REDIS_UNAVAILABLE') {
    status = 503;
    message = 'Security service temporarily unavailable. Please try again.';
    code = 'SERVICE_UNAVAILABLE';
  } else if (err instanceof multer.MulterError) {
    status = err.code === 'LIMIT_FILE_SIZE' ? 413 : 400;
    message = err.code === 'LIMIT_FILE_SIZE' ? 'File is too large' : err.message;
    code = err.code;
  } else if (err.name === 'ValidationError') {
    status = 400;
    message = Object.values(err.errors).map((e) => e.message).join(', ');
  } else if (err.name === 'CastError') {
    status = 400;
    message = `Invalid ${err.path}`;
  } else if (err.code === 11000) {
    status = 409;
    message = 'A record with these details already exists';
    code = 'DUPLICATE';
  } else if (err.type === 'entity.too.large') {
    status = 413;
    message = 'Request body is too large';
  } else if (err.type === 'entity.parse.failed') {
    status = 400;
    message = 'Malformed JSON body';
  }

  if (status >= 500) {
    logger.error(`${req.method} ${req.originalUrl} → ${err.stack || err.message}`);
    if (config.isProduction) message = 'Internal server error';
  }
```

Each branch converts a low-level failure into a meaningful HTTP status. The most important one is the first: when Redis is down, the security layer cannot check OTPs, refresh tokens or revocations, so the request fails with **503** ("fail closed") rather than being let through. In production, details of unexpected (500) errors are logged but not sent to the client.

5.6 Input validation: src/middleware/validate.js

Listing 5.8: The validate middleware (src/middleware/validate.js)

```
const validate = (schemas) => (req, res, next) => {
  const errors = [];
  for (const part of ['params', 'query', 'body']) {
    if (!schemas[part]) continue;
```

```
    const { error, value } = schemas[part].validate(req[part] || {}, {
      abortEarly: false, stripUnknown: false, convert: true,
    });
    if (error) {
      errors.push(...error.details.map((d) => d.message.replace(/"/g, '')));
    } else if (part === 'query') {
      Object.defineProperty(req, 'query', { value, writable: true, configurable: true });
    } else {
      req[part] = value;
    }
  }
  if (errors.length) {
    return res.status(400).json({ success: false, message: 'Validation failed', errors });
  }
  return next();
};

const objectId = () => Joi.string().pattern(/^[a-f\d]{24}$/i).message('must be a valid id');

const password = () => Joi.string()
  .min(8)
  .max(128)
  .pattern(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&#^()\-_=+]).+$/)
```

How it works

Each route declares what it accepts, for example `validate({ body: Joi.object({ doctorId: objectId().required(), ... }) })`.

- `abortEarly:false` reports *every* problem at once, so a form can show all errors together.
- `stripUnknown:false` means unknown fields are an error, not silently dropped: a client cannot sneak in `role:'admin'`.
- `convert:true` turns `"7"` into `7` and ISO strings into dates; the cleaned value replaces the original, so handlers work with trusted, typed data. (`req.query` is a getter in newer Express, hence `defineProperty`.)

5.7 Databases: src/db/

Listing 5.9: GridFS bucket (src/db/mongo.js)

```
/**
 * GridFS stores files larger than a normal document by splitting them into
 * 255 kB chunks (collections 'uploads.files' and 'uploads.chunks'). It ships
 * with the MongoDB driver, so no extra infrastructure is needed (decision D3).
 */
function getBucket() {
  if (!bucket) {
    bucket = new mongoose.mongo.GridFSBucket(mongoose.connection.db, { bucketName: 'uploads' });
  }
  return bucket;
}
```

Listing 5.10: Redis connection that fails loudly (src/db/redis.js)

```
async function connectRedis() {
  if (!redisClient.isOpen) {
    await redisClient.connect();
  }
```

```
  await redisClient.ping();
  return redisClient;
}
```

5.8 Hashing helpers: src/utils/hash.js

These three functions underpin the ledger (Chapter 8).

Listing 5.11: Canonical JSON and hashing (src/utils/hash.js)

```
const sha256 = (value) => crypto.createHash('sha256').update(typeof value === 'string' || Buffer.
    isBuffer(value) ? value : String(value ?? '')).digest('hex');

/**
 * Deterministic JSON: object keys are sorted at every level, so the same data
 * always produces the same string (and therefore the same hash) no matter in
 * which order the fields were assigned.
 */
function canonicalJSON(value) {
  if (value === null || value === undefined) return 'null';
  if (value instanceof Date) return JSON.stringify(value.toISOString());
  if (Array.isArray(value)) return `[${value.map(canonicalJSON).join(',')}]`;
  if (typeof value === 'object') {
    if (typeof value.toHexString === 'function') return JSON.stringify(value.toHexString()); //
    ObjectId
    const keys = Object.keys(value).filter((k) => value[k] !== undefined).sort();
    return `{${keys.map((k) => `${JSON.stringify(k)}:${canonicalJSON(value[k])}`).join(',')}}`;
  }
  return JSON.stringify(value);
}

const hashObject = (value) => sha256(canonicalJSON(value));
```

Why canonical JSON matters

`JSON.stringify({a:1,b:2})` and `JSON.stringify({b:2,a:1})` produce different strings, and therefore different hashes, for the same data. MongoDB does not promise to return fields in the order they were written. Sorting the keys at every level, and writing dates and ObjectIds in one fixed format, makes the hash a function of the *data* only. Without this, verification would report false tampering.

CHAPTER 6

Authentication and Sessions

This chapter covers who a user is (the `User` model), how they prove it (password plus emailed OTP), and how the proof is carried between requests (access and refresh tokens).

6.1 The User model: `src/models/User.js`

One collection holds all four roles. Fields that only make sense for one role live in a sub-document (`patientProfile`, `doctorProfile`, `agentProfile`).

Listing 6.1: User schema, key fields (`src/models/User.js`)

```
const ROLES = ['patient', 'doctor', 'admin', 'insurance_agent'];
// Doctors and insurance agents must be approved by an admin before they can
// touch clinical or financial data. Patients and admins are approved on creation.
const ROLES_NEEDING_APPROVAL = ['doctor', 'insurance_agent'];

const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  fullName: { type: String, trim: true, required: true },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: ROLES, required: true, default: 'patient' },
  // ... phone, dateOfBirth, gender, address, patientProfile, doctorProfile, agentProfile
  approvalStatus: { type: String, enum: ['pending', 'approved', 'rejected'], default: 'approved'
    },
  approvalNote: { type: String, default: '' },

  // Identity of this user inside the hash-chain ledger (was `walletAddress`).
  ledgerAddress: { type: String, index: true },

  isVerified: { type: Boolean, default: false }, // proven control of the email (first OTP login)
  isActive: { type: Boolean, default: true },
  lastLogin: { type: Date, default: null },
  failedLoginAttempts: { type: Number, default: 0 },
  accountLockedUntil: { type: Date, default: null },
  refreshTokens: {
    type: [String],
    default: [],
    validate: { validator: (tokens) => tokens.length <= 5, message: 'A user can have at most 5
    refresh tokens' },
  },
}, { timestamps: true });
```

A doctor's `doctorProfile` also holds `availability`: working days (0 = Sunday), start and end hour, and slot length. Appointments are generated from it (Chapter 9).

Listing 6.2: Password hashing and ledger address (`src/models/User.js`)

```
userSchema.pre('save', async function hashPasswordAndAssignLedgerAddress(next) {
  try {
    if (!this.ledgerAddress) {
      this.ledgerAddress = `mc_${sha256(this._id.toString()).slice(0, 40)}`;
    }
    if (this.isModified('passwordHash')) {
      const salt = await bcrypt.genSalt(10);
```

```
      this.passwordHash = await bcrypt.hash(this.passwordHash, salt);
    }
    next();
  } catch (error) {
    next(error);
  }
});
```

How it works

Code that creates or changes a user puts the *plain* password into `passwordHash`; this hook replaces it with a bcrypt hash before anything is written. `isModified` ensures an already-hashed password is not hashed again on an unrelated save. bcrypt is deliberately slow (cost 10, about 2^{10} rounds), so a stolen database cannot be brute-forced quickly. The `ledgerAddress` is a stable pseudonymous identifier derived from the user id, the successor of the paper's blockchain wallet address.

Listing 6.3: Never leaking secrets in JSON (`src/models/User.js`)

```
userSchema.methods.toJSON = function toJSON() {
  const obj = this.toObject();
  delete obj.passwordHash;
  delete obj.failedLoginAttempts;
  delete obj.accountLockedUntil;
  delete obj.refreshTokens;
  delete obj.__v;
  // Hide the other roles' (empty) profile sections. When a partial populate did
  // not select 'role', keep whatever profile fields were explicitly selected.
  if (obj.role) {
    if (obj.role !== 'patient') delete obj.patientProfile;
    if (obj.role !== 'doctor') delete obj.doctorProfile;
    if (obj.role !== 'insurance_agent') delete obj.agentProfile;
  }
  return obj;
};
```

Express calls `toJSON` whenever a user is sent with `res.json`, so no route can accidentally return a password hash. The `if (obj.role)` guard fixes a real bug found during testing: when an appointment populated only `doctorProfile.specialization` (without `role`), the doctor's profile was stripped and the patient dashboard crashed.

6.2 Registration

Listing 6.4: register, core (`src/modules/auth/auth.controller.js`)

```
async function register(req, res) {
  const { email, password, fullName, role, phone, dateOfBirth, gender, address } = req.body;

  if (await User.exists({ email })) {
    throw conflict('An account with this email already exists', 'EMAIL_TAKEN');
  }

  const needsApproval = User.ROLES_NEEDING_APPROVAL.includes(role);
  const user = new User({
    email,
    fullName,
    passwordHash: password, // hashed by the pre-save hook
    role,
    // ...
```

```
    approvalStatus: needsApproval ? 'pending' : 'approved',
  });
  // ... role-specific profile fields
  await user.save();
  await ledgerService.append({
    chain: 'records', entityType: 'user', entityId: user._id, action: 'REGISTER',
    actorId: user._id, actorRole: role, payload: { email: user.email, role, ledgerAddress: user.
    ledgerAddress },
  });

  if (needsApproval) {
    await notificationService.notifyRole('admin', {
      type: 'warning',
      title: 'New account awaiting verification',
      // ...
      page: 'verification',
    });
  }

  emailService.sendWelcome(email, fullName, role, needsApproval); // fire-and-forget by design
```

How it works

The Joi schema on the route only allows `role` to be `patient`, `doctor` or `insurance_agent`; nobody can register as an admin. Doctors must supply a specialization and licence number, agents a company. A doctor or agent account starts `pending`: they can log in and edit their profile, but every clinical or claims endpoint answers 403 `APPROVAL_PENDING` until an admin approves them (Section 7.3). The welcome email is not awaited: if the mail server is slow or down, registration still succeeds.

6.3 Two-step login

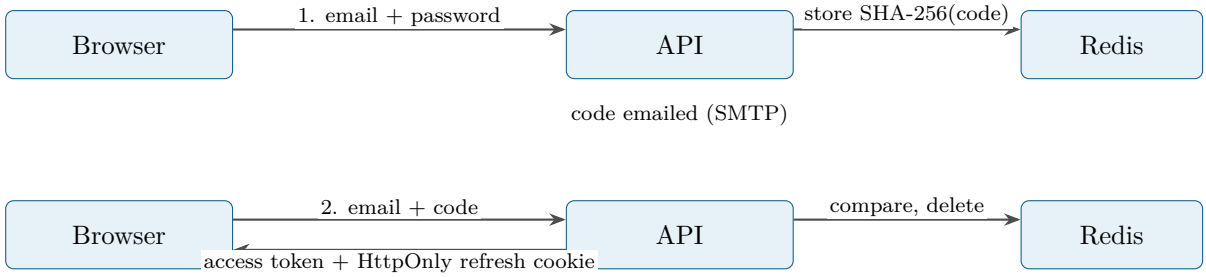


Figure 6.1: Password, then one-time code

6.3.1 Step 1: `POST /api/auth/login/request-otp`

Listing 6.5: requestOtp (src/modules/auth/auth.controller.js)

```
async function requestOtp(req, res) {
  const { email, password } = req.body;
  const user = await User.findOne({ email });

  if (!user) throw unauthorized(INVALID_CREDENTIALS, 'INVALID_CREDENTIALS');
  if (user.isAccountLocked()) {
    throw forbidden('Account is locked after too many failed attempts. Try again later.', '
    ACCOUNT_LOCKED');
  }
```

```
  if (!user.isActive) throw forbidden('Account is deactivated. Contact the hospital administrator
    .', 'ACCOUNT_DEACTIVATED');

  if (!(await user.comparePassword(password))) {
    const failed = await recordFailedAttempt(user);
    return res.status(401).json({
      success: false,
      message: INVALID_CREDENTIALS,
      code: 'INVALID_CREDENTIALS',
      attemptsRemaining: Math.max(0, config.security.maxFailedLogins - failed),
    });
  }

  if (await otpService.getAttempts(email) >= config.otp.maxAttempts) {
    throw tooMany('Too many OTP requests. Try again in 15 minutes.');
  }

  const otp = otpService.generateOTP();
  await otpService.storeOTP(email, otp);
  await emailService.sendOTP(email, otp, user.fullName);
  await otpService.incrementAttempts(email);

  res.json({
    success: true,
    message: 'A login code was sent to your email',
    data: { email, expiresInMinutes: config.otp.expiryMinutes },
  });
}
```

How it works

1. An unknown email and a wrong password give the same message (`INVALID_CREDENTIALS`), so the form cannot be used to find out who has an account.
2. A wrong password increments `failedLoginAttempts`; at `MAX_FAILED_LOGINS` (5) the account is locked for 30 minutes.
3. A per-email counter in Redis limits how many codes can be requested.
4. `sendOTP` is awaited and *throws* on failure: if the email could not be sent the user is told (500), instead of waiting for a code that will never arrive. (A test checks exactly this.)

Listing 6.6: Lockout bookkeeping (`src/modules/auth/auth.controller.js`)

```
async function recordFailedAttempt(user) {
  const updated = await User.findByIdAndUpdate(user._id, { $inc: { failedLoginAttempts: 1 } }, {
    new: true });
  if (updated.failedLoginAttempts >= config.security.maxFailedLogins) {
    updated.accountLockedUntil = new Date(Date.now() + config.security.lockMinutes * 60 * 1000);
    await updated.save({ validateBeforeSave: false });
    logger.warn(`Account locked after ${updated.failedLoginAttempts} failed attempts: ${user.
    email}`);
  }
  await invalidateSession(user._id); // trust score reflects the failure immediately
  return updated.failedLoginAttempts;
}
```

`$inc` is atomic in MongoDB, so two simultaneous wrong guesses are both counted. Recent failures also lower the trust score of any session still open for that user (Section 7.2).

6.3.2 The OTP service: src/services/otpService.js

Listing 6.7: Generating, storing and checking codes (src/services/otpService.js)

```
/** Only the hash is stored: a Redis dump cannot be used to log in. */
hashOTP(otp) {
  return crypto.createHash('sha256').update(String(otp)).digest('hex');
}

/** Cryptographically secure, uniformly random code with no leading zero. */
generateOTP(length = config.otp.length) {
  return crypto.randomInt(10 ** (length - 1), 10 ** length).toString();
}

/** Stores the hashed code with a TTL; a new request overwrites the old code. */
storeOTP(email, otp) {
  return guard('store', () => redisClient.setEx(otpKey(email), config.otp.expiryMinutes * 60,
  this.hashOTP(otp)));
}

/** Single use: a matching code is deleted immediately. */
verifyOTP(email, otp) {
  return guard('verify', async () => {
    const stored = await redisClient.get(otpKey(email));
    if (!stored) return { success: false, message: 'OTP expired or not found' };

    const expected = Buffer.from(stored, 'hex');
    const actual = Buffer.from(this.hashOTP(otp), 'hex');
    // Constant-time comparison: response time does not leak how many characters matched.
    if (expected.length !== actual.length || !crypto.timingSafeEqual(expected, actual)) {
      return { success: false, message: 'Invalid OTP' };
    }
    await redisClient.del(otpKey(email));
    return { success: true, message: 'OTP verified successfully' };
  });
}
```

How it works

- `crypto.randomInt` is a cryptographically secure generator; `Math.random` is predictable and must never be used for codes.
- `setEx` stores the code with a time-to-live: Redis deletes it after 10 minutes without any clean-up job.
- Only the SHA-256 of the code is stored, like a password.
- `timingSafeEqual` always takes the same time, so an attacker cannot learn from response timing how close a guess was.
- `guard` converts any Redis error into `REDIS_UNAVAILABLE`, which becomes a 503: if the OTP store is down, nobody logs in.

6.3.3 Step 2: POST /api/auth/login/verify-otp

Listing 6.8: verifyOtp, success path (src/modules/auth/auth.controller.js)

```
const result = await otpService.verifyOTP(email, otp);
if (!result.success) {
  const otpAttempts = await otpService.incrementAttempts(email);
  const failed = await recordFailedAttempt(user);
```

```
    return res.status(401).json({ /* ... INVALID_OTP, attemptsRemaining */ });
  }

  await otpService.clearAttempts(email);
  user.failedLoginAttempts = 0;
  user.accountLockedUntil = null;
  user.lastLogin = new Date();
  user.isVerified = true;
  await user.save({ validateBeforeSave: false });
  await invalidateSession(user._id);

  const access = await issueSession(res, user);
```

A correct code resets the failure counters and marks the account `isVerified` (the user has proven they control the email address, which raises their trust score). `issueSession` creates the token pair.

6.4 Tokens: src/services/tokenService.js

MediChain uses two tokens with different jobs:

Table 6.1: The two tokens

	Access token	**Refresh token**
Purpose	Sent with every API call	Gets a new access token
Lifetime	15 minutes	7 days
Where the browser keeps it	A JavaScript variable (memory only)	HttpOnly cookie, path `/api/auth`
Signed with	`JWT_SECRET`	`JWT_REFRESH_SECRET`
Revocation	Blacklist of its `jti` in Redis	Valid only while its Redis key exists

Listing 6.9: Issuing tokens (src/services/tokenService.js)

```
/** Short-lived access token. The random jti lets us revoke this exact token. */
generateAccessToken(user) {
  const jti = crypto.randomUUID();
  const token = jwt.sign(
    { userId: user._id.toString(), email: user.email, role: user.role, jti },
    this.config.jwt.secret,
    { expiresIn: this.config.jwt.expiry }
  );
  return { token, jti, expiresIn: this.config.jwt.expiry };
}

/**
 * Long-lived refresh token, signed with a different secret. It is valid only
 * while its Redis key exists, so the server can revoke it at any time.
 */
async generateRefreshToken(user) {
  const userId = user._id.toString();
  const jti = crypto.randomUUID();
  const token = jwt.sign(
    { userId, jti, type: 'refresh' },
    this.config.jwt.refreshSecret,
    { expiresIn: `${this.config.jwt.refreshExpiryDays}d` }
  );
  try {
```

```
      await this.redis.setEx(this.refreshKey(userId, jti), this.refreshTTLSeconds, '1');
    } catch (error) {
      logger.error(`Error storing refresh token: ${error.message}`);
      throw new Error(REDIS_UNAVAILABLE);
    }
    return { token, jti, jtiHash: this.hashJTI(jti) };
  }
```

Listing 6.10: The refresh cookie (src/modules/auth/auth.controller.js)

```
const cookieOptions = () => ({
  httpOnly: true, // JavaScript in the page cannot read it (XSS)
  secure: config.isProduction, // HTTPS-only in production; plain http://localhost works in dev
  sameSite: 'strict', // never sent on cross-site requests (CSRF)
  path: '/api/auth', // only sent to the auth endpoints that need it
  maxAge: config.jwt.refreshExpiryDays * 24 * 60 * 60 * 1000,
});
```

Why the token is not in localStorage

Anything in `localStorage` can be read by any script on the page; one XSS bug would leak every user's long-lived session. Here the long-lived token is in an `HttpOnly` cookie that scripts cannot read, and the short-lived access token exists only in a variable that disappears when the tab closes. A page reload restores the session by calling `/api/auth/refresh-token`, which the browser authenticates with the cookie automatically.

6.5 Refresh-token rotation

Listing 6.11: refreshToken (src/modules/auth/auth.controller.js)

```
async function refreshToken(req, res) {
  const presented = req.cookies?.[REFRESH_COOKIE];
  if (!presented) throw unauthorized('No session. Please log in.', 'NO_REFRESH_TOKEN');

  let decoded;
  try {
    decoded = await tokenService.verifyRefreshToken(presented);
  } catch (error) {
    if (error.message === 'REDIS_UNAVAILABLE') throw error;
    res.clearCookie(REFRESH_COOKIE, { ...cookieOptions(), maxAge: undefined });
    throw unauthorized('Session expired. Please log in again.', 'INVALID_REFRESH_TOKEN');
  }

  const user = await User.findById(decoded.userId);
  if (!user || !user.isActive) {
    await tokenService.revokeRefreshToken(decoded.userId, decoded.jti);
    throw unauthorized('Account deactivated', 'ACCOUNT_DEACTIVATED');
  }

  await tokenService.revokeRefreshToken(decoded.userId, decoded.jti);
  await User.updateOne({ _id: user._id }, { $pull: { refreshTokens: tokenService.hashJTI(decoded.
    jti) } });
  const access = await issueSession(res, user);
  // ... respond with the new access token and user
}
```

How it works

Each refresh token works *once*. Using it deletes its Redis key and issues a new one in a new cookie. If an attacker steals a refresh token and uses it, the real user's next refresh fails (its token is gone), so the theft becomes visible. If the real user refreshes first, the stolen copy is already dead. `verifyRefreshToken` also checks `type === 'refresh'`, so an access token cannot be replayed here.

6.6 Logout, password change and deactivation

Listing 6.12: Revocation on logout (`src/modules/auth/auth.controller.js`) and blacklisting (`tokenService.js`)

```
async function logout(req, res) {
  await tokenService.blacklistAccessToken(req.user);
  // ... verify and revoke the refresh cookie, clear it
}

  /** Blacklists an access token until it would have expired anyway. */
  async blacklistAccessToken(decodedPayload) {
    if (!decodedPayload?.jti || !decodedPayload?.exp) throw new Error('INVALID_TOKEN');
    const ttl = decodedPayload.exp - Math.floor(Date.now() / 1000);
    if (ttl <= 0) return;
    try {
      await this.redis.setEx(`jwt:blacklist:${decodedPayload.jti}`, ttl, '1');
    } catch (error) {
      // ...
      throw new Error(REDIS_UNAVAILABLE);
    }
  }
```

A JWT is valid until it expires; the server cannot "un-sign" it. So logout puts the token's id on a blacklist that `verifyAccessToken` checks on every request. The blacklist entry lives exactly as long as the token would have, so the list never grows without bound.

Changing the password calls `revokeAllRefreshTokens`, which scans Redis for every `refresh:<userId>:*` key and deletes them: every other device is signed out. Deactivating a user in the admin dashboard does the same, plus `invalidateSession` so the next request with a still-valid access token is refused (Chapter 7).

A bug that was fixed

The original controller was a class whose methods were passed to Express as `router.post('/logout', authController.logout)`. Express calls them without an object, so `this` was `undefined` and logout always crashed. The controller is now a set of plain functions with no `this`.

6.7 Redis key space

Table 6.2: What lives in Redis

Key	TTL	Meaning
`otp:<email>`	10 min	SHA-256 of the current login code
`otp:attempts:<email>`	15 min	OTP requests + wrong codes
`refresh:<userId>:<sha256(jti)>`	7 days	This refresh token is still valid
`jwt:blacklist:<jti>`	token's remaining life	This access token was revoked
`session:active:<userId>`	5 min	Cached `isActive`, `isVerified`, `approvalStatus`, failures

CHAPTER 7

The Zero-Trust Layer

"Zero trust" means: never assume a request is legitimate because of an earlier check. MediChain applies four checks to *every* protected request, in the middleware in `src/middleware/auth.js` and the policy engine in `src/services/trustService.js`, and records every decision.

1. **Authentication**: is the token genuine, unexpired and not revoked?
2. **Continuous verification**: is the account still active, and what is its current state?
3. **Trust score**: how much does this particular request look like its legitimate owner?
4. **Authorisation**: does the role, approval status, trust score and (for patient data) consent allow this action?

7.1 `authenticate`

Listing 7.1: authenticate (`src/middleware/auth.js`)

```
async function authenticate(req, res, next) {
  const header = req.headers.authorization || '';
  if (!header.startsWith('Bearer ') || !header.slice(7).trim()) {
    return deny(req, res, 401, 'missing_token', 'Access denied. No token provided. Please log in.
    ');
  }

  let decoded;
  try {
    decoded = await tokenService.verifyAccessToken(header.slice(7).trim());
  } catch (error) {
    if (error.name === 'TokenExpiredError') return deny(req, res, 401, 'token_expired', 'Token
    expired. Please log in again.');
    if (error.message === 'TOKEN_REVOKED') return deny(req, res, 401, 'token_revoked', 'Token has
     been revoked. Please log in again.');
    if (error.message === 'REDIS_UNAVAILABLE') return deny(req, res, 503, 'redis_unavailable', '
    Security service temporarily unavailable. Please try again.');
    return deny(req, res, 401, 'invalid_token', 'Invalid token. Please log in again.');
  }

  let sessionState;
  try {
    sessionState = await loadSessionState(decoded.userId);
  } catch (error) {
    logger.error(`Session state unavailable: ${error.message}`);
    return deny(req, res, 503, 'session_cache_unavailable', 'Security service temporarily
    unavailable. Please try again.', { userId: decoded.userId });
  }

  if (!sessionState || !sessionState.isActive) {
    return deny(req, res, 401, 'account_deactivated', 'Account deactivated. Contact the hospital
    administrator.', { userId: decoded.userId });
  }

  req.user = decoded; // { userId, email, role, jti, iat, exp }
  req.sessionState = sessionState;
```

```
  req.trustScore = trustService.calculateTrustScore(req, decoded, sessionState);

  if (req.trustScore.score < req.trustScore.minScore) {
    return deny(req, res, 403, 'low_trust_score', 'Access denied. Additional verification
    required: please log in again.', {
      userId: decoded.userId, trustScore: req.trustScore.score, body: { trustScore: req.
    trustScore.score },
    });
  }

  await auditService.recordAccessDecision({ req, userId: decoded.userId, decision: 'allow',
    reason: 'authenticated', trustScore: req.trustScore.score });
  return next();
}
```

How it works

Every failure path calls `deny`, which writes the decision to the ledger's access chain before answering, so failed attempts are as visible as successful ones. The reason code (`token_expired`, `token_revoked`, ...) is returned to the client in upper case; the frontend uses `401` to try a silent refresh and `503` to show "security service unavailable".

7.1.1 Continuous verification with a session cache

Listing 7.2: loadSessionState (src/middleware/auth.js)

```
async function loadSessionState(userId) {
  const cached = await redisClient.get(sessionKey(userId));
  if (cached) return JSON.parse(cached);

  const user = await User.findById(userId).select('isActive isVerified failedLoginAttempts
    approvalStatus').lean();
  if (!user) return null;
  const state = {
    isActive: user.isActive,
    isVerified: user.isVerified,
    failedLoginAttempts: user.failedLoginAttempts || 0,
    approvalStatus: user.approvalStatus,
  };
  await redisClient.setEx(sessionKey(userId), SESSION_CACHE_SECONDS, JSON.stringify(state));
  return state;
}
```

A JWT says who the user *was* when it was issued. If an admin deactivates the account a minute later, the token is still cryptographically valid for 14 more minutes. Loading the account state on each request closes that gap. To avoid a MongoDB query per request, the state is cached in Redis for 5 minutes, and every code path that changes it (deactivation, approval, failed login, password change) calls `invalidateSession`, so changes take effect on the very next request.

7.2 The trust score

Listing 7.3: calculateTrustScore (src/services/trustService.js)

```
  calculateTrustScore(req, tokenPayload, sessionState = {}) {
    let score = 100;
    const reasons = [];
    const failedAttempts = Number(sessionState.failedLoginAttempts || 0);
```

```
  const sessionAgeSeconds = tokenPayload?.iat
    ? Math.max(0, Math.floor(Date.now() / 1000) - tokenPayload.iat)
    : null;
  const userAgent = req.get('user-agent') || '';

  if (!sessionState.isVerified) {
    score -= 15;
    reasons.push('user_not_verified');
  }
  if (failedAttempts > 0) {
    score -= Math.min(30, failedAttempts * 6);
    reasons.push('recent_failed_attempts');
  }
  if (!userAgent) {
    score -= 10;
    reasons.push('missing_user_agent');
  }
  if (sessionAgeSeconds !== null && sessionAgeSeconds > 60 * 60) {
    score -= 10;
    reasons.push('older_session');
  }
  if (sessionAgeSeconds !== null && sessionAgeSeconds > 6 * 60 * 60) {
    score -= 15;
    reasons.push('long_lived_session');
  }
  // ... returns { score, reasons, minScore, highSensitivityMinScore, ... }
```

Table 7.1: Trust score signals

Signal	Penalty	Why it matters
Email never verified by OTP	−15	Weaker proof of identity
Recent failed logins	−6 each, max −30	Someone may be guessing this account's password
No User-Agent header	−10	Typical of scripts rather than browsers
Session older than 1 hour	−10	Old credentials are more likely to have leaked
Session older than 6 hours	−15 more	As above

Every request needs at least `MIN_TRUST_SCORE` (50). Medical records and files need `HIGH_SENSITIVITY_TRUST_SCORE` (70). Example: a user with two recent failed logins (−12) whose access token was issued 2 hours ago via refresh (−10) scores 78: allowed everywhere. With five failures (−30) and a 7-hour-old token (−25) the score is 45: every request is refused with "please log in again", and a fresh login (which resets the failures) restores access. The profile page shows each user their current score and the reasons.

7.3 The policy engine

Listing 7.4: evaluatePolicy (src/services/trustService.js)

```
evaluatePolicy(req, allowedRoles, options = {}) {
  const role = req.user?.role;
  const trustScore = req.trustScore?.score ?? 0;
  const minTrustScore = options.minTrustScore ?? config.security.minTrustScore;
  const breakGlassReason = (req.get('x-break-glass-reason') || '').trim();
  const breakGlass = Boolean(options.allowBreakGlass && breakGlassReason.length >= 10 &&
  BREAK_GLASS_ROLES.includes(role));
```

```
  if (breakGlass) {
    return { allowed: true, reason: 'break_glass_allowed', breakGlass: true, breakGlassReason
  };
  }
  if (!allowedRoles.includes(role)) {
    return { allowed: false, status: 403, reason: 'role_denied', message: `Access denied. This
  resource requires the ${allowedRoles.join(' or ')} role.` };
  }
  if (ROLES_NEEDING_APPROVAL.includes(role) && !options.allowUnapproved && req.sessionState?.
  approvalStatus !== 'approved') {
    return { allowed: false, status: 403, reason: 'approval_pending', message: 'Your account is
   awaiting administrator verification.' };
  }
  if (options.requireVerified && !req.sessionState?.isVerified) {
    return { allowed: false, status: 403, reason: 'verification_required', message: 'Access
  denied. Verified account required.' };
  }
  if (trustScore < minTrustScore) {
    return { allowed: false, status: 403, reason: 'low_trust_score', message: `Access denied.
  This resource needs a trust score of ${minTrustScore} (yours is ${trustScore}). Log in again
   to refresh your session.` };
  }
  if (options.timeWindow && !this.isWithinTimeWindow(new Date().getHours(), options.timeWindow)
  ) {
    return { allowed: false, status: 403, reason: 'outside_time_window', message: 'Access
  denied outside the approved access window.' };
  }
  return { allowed: true, reason: 'policy_allowed', breakGlass: false };
}
```

How it works

Routes declare their policy declaratively:

```
const readPolicy = requireRole('patient', 'doctor', {
  minTrustScore: config.security.highSensitivityTrustScore, allowBreakGlass: true,
});
```

`requireRole` (in `auth.js`) calls `evaluatePolicy` and either continues or calls `deny`. The checks run in a fixed order and the first failure wins: role (RBAC), then approval, verification, trust score and time window (attribute-based conditions, ABAC). The result is stored in `req.policy` so the handler knows whether break-glass was used.

7.4 Break-glass emergency access

An unconscious patient in the emergency room cannot approve a consent request. Break-glass lets a doctor (or admin) open the record anyway, under three conditions: the route must allow it (`allowBreakGlass`), the user must be a doctor or admin, and the request must carry a reason of at least 10 characters in the `x-break-glass-reason` header. Break-glass replaces the *consent* check only; the user must still be authenticated.

What happens next is in `consentService.assertCanAccessPatient` (Chapter 10): the access is written to the records chain as `BREAK_GLASS_ACCESS` with the reason, the patient gets a "danger" notification quoting the reason, and every admin is notified to review it. Emergency access is therefore possible, but never silent.

7.5 The audit trail: src/services/auditService.js

Listing 7.5: recordAccessDecision (src/services/auditService.js)

```
async function recordAccessDecision({ req, userId, decision, reason, trustScore, breakGlassReason
    , resource }) {
  try {
    await ledgerService.append({
      chain: 'access',
      entityType: 'access_decision',
      entityId: resource || `${req.method} ${req.originalUrl || req.path}`,
      action: decision === 'allow' ? 'ALLOW' : 'DENY',
      actorId: userId || req.user?.userId || null,
      actorRole: req.user?.role || null,
      meta: {
        reason,
        method: req.method,
        path: (req.originalUrl || req.path || '').split('?')[0],
        ip: req.ip,
        userAgentHash: sha256(req.get('user-agent') || ''),
        trustScore: trustScore ?? req.trustScore?.score ?? null,
        breakGlassReason: breakGlassReason || undefined,
      },
    });
  } catch (error) {
    logger.error(`Audit write failed: ${error.message}`);
  }
}
```

Each decision becomes one block of the `access` chain, so the audit log is itself tamper-evident. The query string is dropped from the path and the User-Agent is stored only as a hash, to avoid putting personal data in a log that can never be edited. Audit failures are logged but do not fail the request: losing one audit line is better than locking every user out. The admin dashboard's *Access Control* page reads this chain through `GET /api/admin/audit`.

CHAPTER 8

The Tamper-Evident Ledger

The ledger is what makes MediChain's records trustworthy. This chapter explains the idea from first principles, then the code that implements it, then how tampering is detected and demonstrated.

8.1 The idea in one picture

A *hash* (SHA-256) turns any data into a 64-character fingerprint. Change one character of the data and the fingerprint changes completely; it is infeasible to find different data with the same fingerprint.

A *hash chain* is a list of entries where each entry stores the fingerprint of the previous entry, and its own fingerprint covers everything including that link:

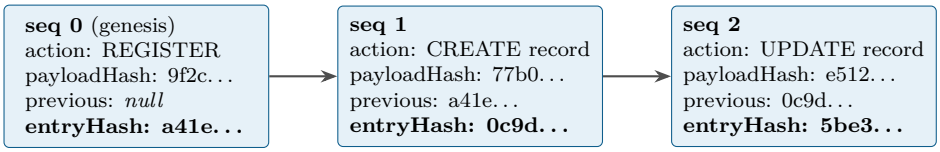


Figure 8.1: Three entries of the records chain

Two kinds of tampering become detectable:

1. **Editing a ledger entry.** If someone changes entry 1's action, its recomputed hash no longer equals its stored `entryHash`. If they also recompute and store the new hash, entry 2's `previousEntryHash` no longer matches. They would have to rewrite every later entry.
2. **Editing the data itself.** Each entry stores the hash of the protected document's content (`payloadHash`) at that moment. If someone edits the medical record directly in MongoDB, the record's current hash no longer equals the hash the ledger last recorded for it, and they cannot fix the ledger without breaking the chain (point 1).

8.2 Two chains

- **records**: every create or change of clinical and financial data (users registered and verified, records, file attachments, consents, appointments, prescriptions, bills, claims, break-glass accesses).
- **access**: every allow or deny decision of the zero-trust layer.

They are independent sequences in the same collection, so the high-volume access log does not slow down reads of a record's history.

8.3 The entry: `src/models/LedgerEntry.js`

Listing 8.1: LedgerEntry schema (src/models/LedgerEntry.js)

```
const ledgerEntrySchema = new mongoose.Schema({
  chain: { type: String, enum: ['records', 'access'], required: true },
  seq: { type: Number, required: true }, // position in the chain, starting at 0
  entityType: { type: String, required: true },
  entityId: { type: String, default: null },
  action: { type: String, required: true },
  actorId: { type: String, default: null },
  actorRole: { type: String, default: null },
  payloadHash: { type: String, default: null }, // SHA-256 of the entity's content at this moment
  meta: { type: mongoose.Schema.Types.Mixed, default: {} },
  timestamp: { type: Date, required: true },
  previousEntryHash: { type: String, default: null }, // null only for the genesis entry
  entryHash: { type: String, required: true },
}, { versionKey: false });

// The unique (chain, seq) index makes forking impossible: two writers that read
// the same "last entry" both try to insert seq N, and the second one fails.
ledgerEntrySchema.index({ chain: 1, seq: 1 }, { unique: true });
```

The application has no route that updates or deletes ledger entries. The only code that writes them is `ledgerService.append`.

8.4 Writing an entry: src/services/ledgerService.js

Listing 8.2: computeEntryHash and appendOnce (src/services/ledgerService.js)

```
function computeEntryHash(entry) {
  return sha256(canonicalJSON({
    chain: entry.chain,
    seq: entry.seq,
    entityType: entry.entityType,
    entityId: entry.entityId ?? null,
    action: entry.action,
    actorId: entry.actorId ?? null,
    actorRole: entry.actorRole ?? null,
    payloadHash: entry.payloadHash ?? null,
    meta: entry.meta || {},
    timestamp: new Date(entry.timestamp).toISOString(),
    previousEntryHash: entry.previousEntryHash ?? null,
  }));
}

async function appendOnce({ chain, entityType, entityId, action, actorId, actorRole, payload,
    payloadHash, meta }) {
  const last = await LedgerEntry.findOne({ chain }).sort({ seq: -1 }).select('seq entryHash').
    lean();
  const entry = {
    chain,
    seq: last ? last.seq + 1 : 0,
    entityType,
    entityId: entityId ? String(entityId) : null,
    action,
    actorId: actorId ? String(actorId) : null,
    actorRole: actorRole || null,
    payloadHash: payloadHash || (payload !== undefined ? hashObject(payload) : null),
    // JSON round-trip drops undefined values, which MongoDB would otherwise
    // store as null and so change the hash on read-back.
    meta: meta ? JSON.parse(JSON.stringify(meta)) : {},
    timestamp: new Date(),
    previousEntryHash: last ? last.entryHash : null,
```

```
  };
  entry.entryHash = computeEntryHash(entry);
  return LedgerEntry.create(entry);
}
```

How it works

1. Read the current head of the chain (highest `seq`).
2. Build the new entry with `seq = head.seq + 1` and `previousEntryHash = head.entryHash`.
3. Hash the entity's content (`payload`) with canonical JSON (Section 5), unless a hash was supplied (files supply the SHA-256 of their bytes).
4. Hash every field of the entry, store it as `entryHash`, insert.

The `meta` round-trip is a subtle fix: an `undefined` value would be hashed as "absent" but stored by MongoDB as `null`, so the recomputed hash on read-back would differ and the chain would appear broken.

8.4.1 Concurrency: two requests at once

If two requests read the same head and both insert `seq = N`, the chain would fork. Two mechanisms prevent this:

Listing 8.3: Serialised appends with retry (src/services/ledgerService.js)

```
const queues = new Map();

function enqueue(chain, task) {
  const previous = queues.get(chain) || Promise.resolve();
  const next = previous.then(task, task);
  queues.set(chain, next.catch(() => {}));
  return next;
}

function append(params) {
  return enqueue(params.chain, async () => {
    for (let attempt = 0; attempt < 5; attempt += 1) {
      try {
        return await appendOnce(params);
      } catch (error) {
        if (error.code !== 11000) throw error; // not a sequence collision
      }
    }
    throw new Error('LEDGER_APPEND_CONFLICT');
  });
}
```

- **Within one process**, appends to a chain form a promise queue: each waits for the previous one, so they read the head one after another.
- **Across processes** (several API instances), the unique `(chain, seq)` index makes the second insert fail with duplicate-key error 11000; the writer re-reads the head and retries.

8.4.2 The shortcut used by every module

Listing 8.4: recordEvent (src/services/ledgerService.js)

```
function recordEvent({ entityType, entity, action, actor, meta, payloadHash }) {
  return append({
    chain: 'records',
    entityType,
    entityId: entity._id,
    action,
    actorId: actor ? actor.userId || actor._id : null,
    actorRole: actor ? actor.role : 'system',
    payload: typeof entity.ledgerPayload === 'function' ? entity.ledgerPayload() : undefined,
    payloadHash,
    meta,
  });
}
```

Each protected model defines `ledgerPayload()`: the fields whose integrity matters. For a medical record:

Listing 8.5: What the ledger protects for a record (src/models/MedicalRecord.js)

```
medicalRecordSchema.methods.ledgerPayload = function ledgerPayload() {
  return {
    patientId: this.patientId.toString(),
    doctorId: this.doctorId ? this.doctorId.toString() : null,
    recordType: this.recordType,
    title: this.title,
    description: this.description,
    diagnosis: this.diagnosis,
    findings: this.findings,
    attachments: this.attachments.map((a) => ({ fileId: a.fileId, sha256: a.sha256 })),
    version: this.version,
  };
};
```

Bookkeeping fields such as `updatedAt` or `ledgerEntryId` are left out on purpose: they change for harmless reasons and would cause false alarms. The same pattern exists on `Appointment`, `AccessGrant`, `Prescription`, `Bill` and `Claim`.

8.5 Verifying

8.5.1 Walking a chain

Listing 8.6: verifyChain (src/services/ledgerService.js)

```
async function verifyChain(chain) {
  const cursor = LedgerEntry.find({ chain }).sort({ seq: 1 }).lean().cursor();
  let previous = null;
  let checked = 0;

  for await (const entry of cursor) {
    const expectedSeq = previous ? previous.seq + 1 : 0;
    if (entry.seq !== expectedSeq) {
      return { chain, valid: false, entriesChecked: checked, brokenAtSeq: expectedSeq, problem: '
    missing_entry' };
    }
    if (entry.previousEntryHash !== (previous ? previous.entryHash : null)) {
      return { chain, valid: false, entriesChecked: checked, brokenAtSeq: entry.seq, problem: '
    broken_link' };
    }
    if (computeEntryHash(entry) !== entry.entryHash) {
```

```
      return { chain, valid: false, entriesChecked: checked, brokenAtSeq: entry.seq, problem: '
    entry_modified' };
    }
    previous = entry;
    checked += 1;
  }

  return { chain, valid: true, entriesChecked: checked, headHash: previous ? previous.entryHash :
     null };
}
```

A database cursor streams entries one by one, so even a large chain is verified in constant memory. The three checks detect a deleted entry (a gap in `seq`), a re-ordered or re-linked entry, and an edited entry.

8.5.2 Checking a document against the ledger

Listing 8.7: verifyEntity (src/services/ledgerService.js)

```
async function verifyEntity(entityType, entity) {
  const latest = await LedgerEntry.findOne({
    chain: 'records', entityType, entityId: String(entity._id), payloadHash: { $ne: null },
    action: { $nin: ['FILE_ATTACHED', 'VIEW', 'BREAK_GLASS_ACCESS'] },
  }).sort({ seq: -1 }).lean();

  if (!latest) {
    return { entityId: String(entity._id), matches: false, reason: 'no_ledger_entry' };
  }
  const currentHash = hashObject(entity.ledgerPayload());
  return {
    entityId: String(entity._id),
    matches: currentHash === latest.payloadHash,
    currentHash,
    ledgerHash: latest.payloadHash,
    ledgerSeq: latest.seq,
  };
}
```

The most recent content-bearing entry for the document holds the hash of what the document *should* contain. Recompute the document's hash now; if they differ, it was changed without going through the API. A document with no ledger entry at all (inserted directly into the database) is also reported.

8.5.3 The full audit: `GET /api/ledger/verify`

Listing 8.8: Full integrity audit (src/modules/ledger/ledger.routes.js)

```
const PROTECTED = [
  { entityType: 'record', model: MedicalRecord, label: 'Medical records', describe: (d) => d.
    title },
  { entityType: 'prescription', model: Prescription, label: 'Prescriptions', describe: (d) => `${
    d.medication} ${d.dosage}` },
  { entityType: 'consent', model: AccessGrant, label: 'Consents', describe: (d) => `consent ${d.
    status}` },
  { entityType: 'appointment', model: Appointment, label: 'Appointments', describe: (d) => d.
    startTime.toISOString() },
  { entityType: 'claim', model: Claim, label: 'Insurance claims', describe: (d) => d.claimNumber
    },
  { entityType: 'bill', model: Bill, label: 'Bills', describe: (d) => d.billNumber },
];
```

```
router.get('/verify', asyncHandler(async (req, res) => {
  const started = Date.now();
  const chains = await Promise.all(['records', 'access'].map((c) => ledgerService.verifyChain(c))
    );

  const entities = [];
  for (const { entityType, model, label, describe } of PROTECTED) {
    const docs = await model.find();
    const results = await Promise.all(docs.map((doc) => ledgerService.verifyEntity(entityType,
    doc)));
    const tampered = results.map((r, i) => ({ ...r, name: describe(docs[i]) })).filter((r) => !r.
    matches);
    entities.push({
      entityType,
      label,
      checked: results.length,
      tampered: tampered.map((t) => ({ entityId: t.entityId, name: t.name, reason: t.reason || '
    content_changed', ledgerSeq: t.ledgerSeq })),
    });
  }

  const valid = chains.every((c) => c.valid) && entities.every((e) => e.tampered.length === 0);
  res.json({ success: true, data: { valid, chains, entities, checkedAt: new Date(), durationMs:
    Date.now() - started } });
}));
```

This is what the admin's **Verify ledger** button calls, and what `npm run ledger:verify` runs from the command line. For one record, `GET /api/records/:id/verify` additionally re-reads every attached file from GridFS and compares the SHA-256 of its bytes with the hash in the ledger (Chapter 10).

8.6 Demonstrating tamper detection

`scripts/tamper-demo.js` plays a malicious insider with direct database access:

Listing 8.9: The insider edit (`scripts/tamper-demo.js`)

```
  const record = await MedicalRecord.findOne({ title: 'Stage 1 essential hypertension' });
  if (!record) throw new Error('Seeded record not found. Run "npm run seed -- --reset" first.');
  fs.writeFileSync(STATE, JSON.stringify({ recordId: record._id.toString(), original: record.
    diagnosis }));
  // updateOne on the raw collection: no API, no validation, no ledger entry. What an insider
    with DB access could do.
  await MedicalRecord.collection.updateOne({ _id: record._id }, { $set: { diagnosis: 'No
    hypertension. Patient healthy.' } });
```

Listing 8.10: Running the demonstration

```
npm run demo:tamper            # change the diagnosis directly in MongoDB
npm run ledger:verify          # → record ... TAMPERED: <id>  ...  TAMPERING DETECTED (exit 1)
npm run demo:tamper -- --restore
npm run ledger:verify          # → LEDGER VALID
npm run demo:tamper -- --ledger    # rewrite a ledger entry → chain "records": BROKEN (
    entry_modified)
```

The same result appears in the admin dashboard (Ledger → Verify ledger) and on the patient's record (Verify integrity). The integration test in `tests/integration/records.test.js` automates exactly this scenario:

Listing 8.11: Tamper detection test (tests/integration/records.test.js)

```
test('editing a record directly in MongoDB (bypassing the API) is detected', async () => {
  const w = await world();
  await grantAccess(w);
  const created = await newRecord(w.doctor, w.patientUser._id);
  const id = created.body.data._id;

  await MedicalRecord.updateOne({ _id: id }, { $set: { diagnosis: 'Healthy, no hypertension' }
  });

  const verify = await as(w.doctor).get(`/api/records/${id}/verify`);
  expect(verify.body.data.intact).toBe(false);
  expect(verify.body.data.content.matches).toBe(false);

  const audit = await as(w.admin).get('/api/ledger/verify');
  expect(audit.body.data.valid).toBe(false);
  expect(audit.body.data.chains.every((c) => c.valid)).toBe(true); // the chain itself is
  intact
  const recordCheck = audit.body.data.entities.find((e) => e.entityType === 'record');
  expect(recordCheck.tampered.map((t) => t.entityId)).toEqual([id]);
});
```

8.7 Limits and how to close them

A hash chain proves that history is *internally consistent*. An attacker with full write access to MongoDB *and* the patience to recompute every hash after the edited point could produce a new, consistent chain. Two inexpensive measures defeat that:

- **Anchoring.** Periodically publish the head hash of each chain (`GET /api/ledger/stats` returns it) somewhere the attacker cannot change: a signed email to auditors, a public timestamping service, or a public blockchain transaction. A rewritten chain will not reproduce a published head hash.
- **Separation of duties.** Run the API with a MongoDB user that may insert into `ledgerentries` but not update or delete.

Neither is implemented yet; both are listed in Chapter 17. Moving to Hyperledger Fabric later would replace `ledgerService` only: every caller already goes through `append` and `recordEvent`.

CHAPTER 9

Appointments

What it does. Patients see a doctor's free slots and book one. Doctors confirm, complete or mark no-show. Patients can reschedule; either side can cancel. Completing a visit raises the bill.

Files. `src/models/Appointment.js`, `src/modules/appointments/slots.js`, `src/modules/appointments/appointments.routes.js`, `src/modules/users/users.routes.js` (available slots).

9.1 Generating slots from availability

Doctors do not create slots one by one. Each doctor has an availability rule (for example Monday–Friday, 9:00–17:00, 30-minute slots) and slots are computed from it.

Listing 9.1: buildDaySlots and isValidSlot (`src/modules/appointments/slots.js`)

```
function buildDaySlots(availability, dateString, maxAdvanceDays, now = new Date()) {
  const [year, month, day] = dateString.split('-').map(Number);
  const date = new Date(year, month - 1, day);
  if (Number.isNaN(date.getTime())) return [];

  const latest = new Date(now);
  latest.setDate(latest.getDate() + maxAdvanceDays);
  if (date > latest) return [];
  if (!availability.days.includes(date.getDay())) return [];

  const slots = [];
  const stepMs = availability.slotMinutes * 60 * 1000;
  const dayEnd = new Date(year, month - 1, day, availability.endHour, 0, 0, 0);
  for (let t = new Date(year, month - 1, day, availability.startHour, 0, 0, 0); t.getTime() +
    stepMs <= dayEnd.getTime(); t = new Date(t.getTime() + stepMs)) {
    if (t > now) slots.push({ start: t, end: new Date(t.getTime() + stepMs) });
  }
  return slots;
}

/** True when `start` is exactly one of the doctor's slot boundaries. */
function isValidSlot(availability, start, maxAdvanceDays, now = new Date()) {
  const dateString = `${start.getFullYear()}-${String(start.getMonth() + 1).padStart(2, '0')}-${
    String(start.getDate()).padStart(2, '0')}`;
  return buildDaySlots(availability, dateString, maxAdvanceDays, now).some((s) => s.start.getTime
    () === start.getTime());
}
```

How it works

- Days outside the doctor's working days, or beyond the hospital's booking window (`maxAdvanceBookingDays`, an admin setting), have no slots.
- A slot is included only if it ends by `endHour`, and only if it is still in the future.
- `now` is a parameter so the unit tests can fix the clock.
- `isValidSlot` is what the booking endpoint uses: a client cannot book 9:07 or 3 a.m. by crafting a request, only a real slot.

`GET /api/doctors/:doctorId/available-slots?date=YYYY-MM-DD` builds the day's slots, then looks up appointments that already hold one of them, and marks each slot `available: true/false`. The patient's booking dialog shows taken slots greyed out.

9.2 Double-booking prevention

Hiding taken slots in the interface is not enough: two patients can open the dialog at the same moment and click the same slot. The guarantee is enforced by the database:

Listing 9.2: Partial unique index (src/models/Appointment.js)

```
  // True while the appointment occupies its slot. Cancelling sets it to false,
  // which frees the slot for someone else.
  holdsSlot: { type: Boolean, default: true },
}, { timestamps: true });

// Double-booking prevention at the database level: among appointments that
// hold their slot, a doctor can have only one per start time. Even two
// requests racing each other cannot both succeed.
appointmentSchema.index(
  { doctorId: 1, startTime: 1 },
  { unique: true, partialFilterExpression: { holdsSlot: true } }
);

appointmentSchema.pre('validate', function syncHoldsSlot(next) {
  this.holdsSlot = ACTIVE_STATUSES.includes(this.status);
  next();
});
```

How it works

A *unique index* makes MongoDB reject a second document with the same `(doctorId, startTime)`. A *partial* index applies only to documents matching a filter: here, those with `holdsSlot: true`. Pending, confirmed and completed appointments hold their slot; cancelled and no-show ones do not. So cancelling an appointment frees the slot automatically, while a race between two bookings always ends with exactly one success. The `pre('validate')` hook keeps `holdsSlot` in step with `status` on every save.

9.3 Booking: `POST /api/appointments`

Listing 9.3: Booking a slot (src/modules/appointments/appointments.routes.js)

```
router.post('/', requireRole('patient'),
  validate({ body: Joi.object({
    doctorId: objectId().required(),
    startTime: Joi.date().iso().greater('now').required(),
    type: Joi.string().valid('consultation', 'follow_up', 'checkup').default('consultation'),
    reason: Joi.string().trim().min(3).max(500).required(),
  }) }),
  asyncHandler(async (req, res) => {
    const { doctorId, startTime, type, reason } = req.body;
    const doctor = await User.findOne({ _id: doctorId, role: 'doctor', approvalStatus: 'approved'
    , isActive: true });
    if (!doctor) throw notFound('Doctor not found');

    const settings = await HospitalSettings.getSingleton();
    const start = new Date(startTime);
```

```
    if (!isValidSlot(doctor.doctorProfile.availability, start, settings.maxAdvanceBookingDays)) {
      throw badRequest('That time is not one of the doctor\'s available slots');
    }

    let appointment;
    try {
      appointment = await Appointment.create({
        patientId: req.user.userId,
        doctorId,
        startTime: start,
        endTime: new Date(start.getTime() + doctor.doctorProfile.availability.slotMinutes *
    60000),
        type,
        reason,
      });
    } catch (error) {
      if (error.code === 11000) throw conflict('This slot has just been booked. Please choose
    another time.', 'SLOT_TAKEN');
      throw error;
    }

    await recordAppointmentEvent(appointment, 'CREATE', req);
    await notificationService.notify(doctorId, {
      type: 'info', title: 'New appointment request',
      message: `A patient requested a ${type.replace('_', '-')} on ${fmt(start)}: "${reason}"`,
    page: 'appointments',
    });
    // ... 201 with the populated appointment
```

How it works

Only verified, active doctors can be booked. The duplicate-key error (11000) from the unique index is translated into a friendly 409 `SLOT_TAKEN`; the frontend shows the message and reloads the slot grid. Every booking is a ledger entry and a notification for the doctor.

9.4 Status changes, rescheduling and cancelling

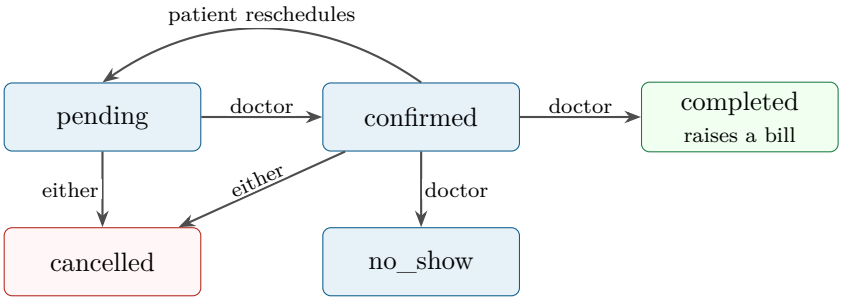


Figure 9.1: Appointment states

Listing 9.4: Allowed doctor transitions (`appointments.routes.js`)

```
const DOCTOR_TRANSITIONS = { pending: ['confirmed'], confirmed: ['completed', 'no_show'] };
// ...
      const allowed = DOCTOR_TRANSITIONS[appointment.status] || [];
      if (!allowed.includes(req.body.status)) {
        throw conflict(`Cannot change an appointment from "${appointment.status}" to "${req.body.
    status}"`, 'INVALID_TRANSITION');
      }
```

```
    if (req.body.status === 'completed' && appointment.startTime > new Date()) {
      throw conflict('An appointment cannot be completed before it starts', 'INVALID_TRANSITION
  ');
    }
    appointment.status = req.body.status;
    if (req.body.notes !== undefined) appointment.notes = req.body.notes;
    await appointment.save();
    await recordAppointmentEvent(appointment, req.body.status.toUpperCase(), req);
    // ... notify the patient
    if (req.body.status === 'completed') {
      const doctor = await User.findById(appointment.doctorId);
      await createBillForAppointment(appointment, doctor, req.user);
    }
```

- **Who may do what** is checked on the server: only the doctor changes status (and only once approved), only the patient reschedules.
- **Rescheduling** (`PATCH` with `startTime`) re-runs `isValidSlot`, moves the appointment, sets it back to *pending* (the doctor must confirm the new time) and catches the same 11000 error if the new slot was just taken.
- **Cancelling** (`DELETE`) is allowed for pending and confirmed appointments; the record stays (with who cancelled and why) and the slot is freed through `holdsSlot`.
- An appointment that is not yours returns **404**, not 403, so appointment ids cannot be probed.

CHAPTER 10

Medical Records, Consent and Files

This is the heart of the system: who may see a patient's medical data, how records are written and amended, and how attached files are stored and proven unchanged.

10.1 Consent model

A patient's consent is an `AccessGrant` document linking one patient and one doctor, with a status and an expiry date.

Listing 10.1: AccessGrant (src/models/AccessGrant.js)

```
const accessGrantSchema = new mongoose.Schema({
  patientId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  doctorId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  status: { type: String, enum: ['requested', 'active', 'denied', 'revoked'], default: 'requested
    ' },
  purpose: { type: String, trim: true, default: '' },
  requestedBy: { type: String, enum: ['doctor', 'patient'], required: true },
  grantedAt: { type: Date, default: null },
  expiresAt: { type: Date, default: null },
  revokedAt: { type: Date, default: null },
  ledgerEntryId: { type: String, default: null },
}, { timestamps: true });

/** The status the user sees: an active grant past its expiry shows as "expired". */
accessGrantSchema.methods.effectiveStatus = function effectiveStatus(now = new Date()) {
  if (this.status === 'active' && (!this.expiresAt || this.expiresAt <= now)) return 'expired';
  return this.status;
};
```

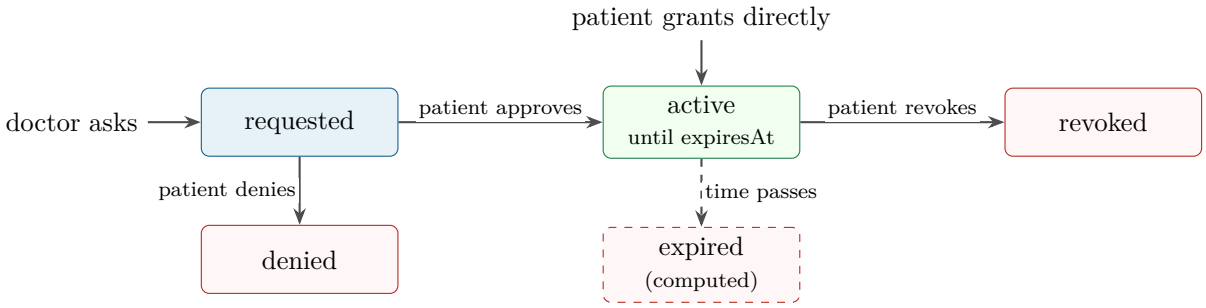


Figure 10.1: Consent lifecycle

How it works

"Expired" is never written to the database: it is computed from `expiresAt` every time. That way a grant expires at exactly the right moment, with no background job. Every transition (`REQUEST_ACCESS`, `GRANT_ACCESS`, `APPROVE`, `DENY`, `REVOKE`) is a ledger entry, and the other party is notified.

Listing 10.2: A patient grants access directly (src/modules/consents/consents.routes.js)

```
    const settings = await HospitalSettings.getSingleton();
    const days = req.body.durationDays || settings.defaultConsentDays;

    // Replace any pending request or existing active grant with this one.
    await AccessGrant.updateMany(
      { patientId: req.user.userId, doctorId: doctor._id, status: { $in: ['requested', 'active']
    } },
      { $set: { status: 'revoked', revokedAt: new Date() } }
    );
    const now = new Date();
    const grant = await AccessGrant.create({
      patientId: req.user.userId, doctorId: doctor._id, status: 'active', purpose: req.body.
    purpose,
      requestedBy: 'patient', grantedAt: now, expiresAt: new Date(now.getTime() + days *
    86400000),
    });
    await ledger(grant, 'GRANT_ACCESS', req);
```

10.2 The consent gate: src/services/consentService.js

Every module that returns patient data (records, files, vitals, prescriptions) calls one function:

Listing 10.3: assertCanAccessPatient (src/services/consentService.js)

```
async function hasActiveGrant(doctorId, patientId, now = new Date()) {
  const grant = await AccessGrant.exists({
    doctorId, patientId, status: 'active', expiresAt: { $gt: now },
  });
  return Boolean(grant);
}

async function assertCanAccessPatient(req, patientId, { purpose = 'view medical data' } = {}) {
  const { userId, role } = req.user;
  const pid = String(patientId);

  if (role === 'patient') {
    if (userId === pid) return { via: 'owner' };
    throw forbidden('Patients can only access their own data.', 'NOT_OWNER');
  }

  if (role === 'doctor' && await hasActiveGrant(userId, pid)) {
    return { via: 'consent' };
  }

  if (req.policy?.breakGlass) {
    await ledgerService.append({
      chain: 'records',
      entityType: 'patient',
      entityId: pid,
      action: 'BREAK_GLASS_ACCESS',
      actorId: userId,
      actorRole: role,
      meta: { reason: req.policy.breakGlassReason, purpose, path: req.originalUrl },
    });
    await notificationService.notify(pid, {
      type: 'danger',
      title: 'Emergency access to your records',
      message: `A ${role} accessed your records under emergency (break-glass) access. Reason
    given: "${req.policy.breakGlassReason}". This access is logged in the ledger.`,
      page: 'permissions',
    });
```

```
    await notificationService.notifyRole('admin', { /* ... 'Break-glass access used' */ });
    return { via: 'break_glass' };
  }

  throw forbidden('No active consent from this patient. Request access, or use emergency access
    if this is an emergency.', 'CONSENT_REQUIRED');
}
```

How it works

- A patient sees only their own data.
- A doctor needs a grant that is `active` *and* not past `expiresAt`, checked in the same query.
- Admins run the system but do not treat patients: they have no consent path, only audited break-glass.
- The return value says *how* access was obtained. The records list passes it to the frontend (`meta.accessVia`), which shows a red emergency banner when it is `break_glass`.

10.3 Records: src/modules/records/records.routes.js

Listing 10.4: Record policies and creation (records.routes.js)

```
// Medical data is high-sensitivity: it needs the higher trust bar, and doctors
// or admins may override consent in an emergency (break-glass, audited).
const readPolicy = requireRole('patient', 'doctor', {
  minTrustScore: config.security.highSensitivityTrustScore, allowBreakGlass: true,
});
const writePolicy = requireRole('patient', 'doctor', { minTrustScore: config.security.
    highSensitivityTrustScore });
// ...
    if (role === 'patient') {
      patientId = userId;
      if (req.body.recordType !== 'patient_upload') throw forbidden('Patients can only add their
    own uploaded documents');
    } else {
      if (!patientId) throw badRequest('patientId is required');
      if (req.body.recordType === 'patient_upload') throw badRequest('Choose a clinical record
    type');
      await consentService.assertCanAccessPatient(req, patientId, { purpose: 'create record' });
    }

    const record = await MedicalRecord.create({
      ...req.body,
      patientId,
      doctorId: role === 'doctor' ? userId : null,
    });
    const entry = await ledgerService.recordEvent({ entityType: 'record', entity: record, action:
     'CREATE', actor: req.user });
    record.ledgerEntryId = entry._id.toString();
    await record.save();
```

Note that the *write* policy does not allow break-glass: emergency access is for reading a history, not for writing records without consent.

10.3.1 Amending a record

Listing 10.5: Versioned amendment (records.routes.js)

```
    const record = await loadRecordWithAccess(req);
    if (!record.doctorId || record.doctorId.toString() !== req.user.userId) {
      throw forbidden('Only the doctor who wrote this record can amend it');
    }
    Object.assign(record, req.body);
    record.version += 1;
    const entry = await ledgerService.recordEvent({
      entityType: 'record', entity: record, action: 'UPDATE', actor: req.user, meta: { version:
    record.version },
    });
    record.ledgerEntryId = entry._id.toString();
    await record.save();
```

Records are never edited silently: the version number goes up and a new `UPDATE` entry, containing the hash of the new content, is chained. The ledger history (`GET /api/records/:id/ledger-history`) therefore shows every version, who made it and when, with the actor's name resolved for display.

10.4 Files: GridFS with hashes in the ledger

Listing 10.6: Upload limits (records.routes.js)

```
const ALLOWED_TYPES = ['application/pdf', 'image/png', 'image/jpeg', 'text/plain'];
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: config.files.maxBytes, files: 1 },
  fileFilter: (req, file, cb) => (ALLOWED_TYPES.includes(file.mimetype)
    ? cb(null, true)
    : cb(badRequest('Only PDF, PNG, JPEG and plain-text files are accepted'))),
});
```

Listing 10.7: Storing a file and anchoring its hash (records.routes.js)

```
    const sha256 = crypto.createHash('sha256').update(req.file.buffer).digest('hex');
    const fileId = await new Promise((resolve, reject) => {
      const stream = getBucket().openUploadStream(req.file.originalname, {
        metadata: { recordId: record._id.toString(), patientId: record.patientId.toString(),
    sha256, contentType: req.file.mimetype, uploadedBy: req.user.userId },
      });
      Readable.from(req.file.buffer).pipe(stream).on('error', reject).on('finish', () => resolve(
    stream.id.toString()));
    });

    record.attachments.push({
      fileId, filename: req.file.originalname, contentType: req.file.mimetype, size: req.file.
    size, sha256, uploadedBy: req.user.userId,
    });
    await ledgerService.append({
      chain: 'records', entityType: 'file', entityId: fileId, action: 'FILE_ATTACHED',
      actorId: req.user.userId, actorRole: req.user.role, payloadHash: sha256,
      meta: { recordId: record._id.toString(), filename: req.file.originalname, size: req.file.
    size },
    });
    // The record's own content (its attachment list) changed too, so chain a new version.
    record.version += 1;
    const entry = await ledgerService.recordEvent({ entityType: 'record', entity: record, action:
     'UPDATE', actor: req.user, meta: { attached: fileId } });
```

How it works

1. multer accepts one file, checks its type and size, and keeps it in memory (at most 10 MB).
2. The SHA-256 of the bytes is computed *before* storage.
3. The bytes stream into GridFS, which splits them into 255 kB chunks.
4. Two ledger entries: `FILE_ATTACHED` with the file's hash (the place where the paper put the IPFS content hash), and a record `UPDATE`, because the record's attachment list is part of its protected content.

10.4.1 Downloading

Listing 10.8: Consent-gated download (src/modules/records/files.routes.js)

```
const record = await MedicalRecord.findOne({ 'attachments.fileId': req.params.fileId });
if (!record) throw notFound('File not found');
await consentService.assertCanAccessPatient(req, record.patientId, { purpose: 'download file'
 });

const attachment = record.attachments.find((a) => a.fileId === req.params.fileId);
res.set({
  'Content-Type': attachment.contentType,
  'Content-Length': attachment.size,
  'Content-Disposition': `inline; filename="${attachment.filename.replace(/"/g, '')}"`,
  'X-Content-SHA256': attachment.sha256,
  'Cache-Control': 'private, no-store',
});
getBucket().openDownloadStream(new mongoose.Types.ObjectId(req.params.fileId))
  .on('error', () => { if (!res.headersSent) res.status(404).end(); else res.end(); })
  .pipe(res);
```

A file is reachable only through its record, so it inherits exactly the record's access rules. `no-store` keeps medical files out of shared caches. The expected hash travels in `X-Content-SHA256` so a client can check the download itself.

10.4.2 Verifying a record and its files

Listing 10.9: Per-record integrity check (records.routes.js)

```
const record = await loadRecordWithAccess(req);
const content = await ledgerService.verifyEntity('record', record);

const files = [];
for (const attachment of record.attachments) {
  const hash = crypto.createHash('sha256');
  try {
    await new Promise((resolve, reject) => {
      getBucket().openDownloadStream(new mongoose.Types.ObjectId(attachment.fileId))
        .on('data', (chunk) => hash.update(chunk)).on('error', reject).on('end', resolve);
    });
    const actual = hash.digest('hex');
    files.push({ fileId: attachment.fileId, filename: attachment.filename, matches: actual ===
  attachment.sha256, expected: attachment.sha256, actual });
  } catch (error) {
    files.push({ fileId: attachment.fileId, filename: attachment.filename, matches: false,
  error: 'file_missing' });
  }
```

```
  }
  const intact = content.matches && files.every((f) => f.matches);
```

The *Verify integrity* button on a record (patient and doctor dashboards) calls this. It re-hashes the record's content and every stored file, streaming the bytes so large files do not fill memory, and reports which part, if any, no longer matches the ledger.

CHAPTER 11

Prescriptions, Vital Signs and IoT Devices

11.1 Prescriptions

Files. src/models/Prescription.js, src/modules/prescriptions/prescriptions.routes.js.

A doctor with the patient's consent prescribes a medication with dosage, frequency, instructions, dates and a number of allowed refills. The patient sees active and past medications and can request a refill; the doctor approves or denies it.

Listing 11.1: Refill request rules (prescriptions.routes.js)

```
router.post('/:id/refill-request', requireRole('patient'), validate(idParam), asyncHandler(async
    (req, res) => {
  const rx = await Prescription.findOne({ _id: req.params.id, patientId: req.user.userId });
  if (!rx) throw notFound('Prescription not found');
  if (rx.status !== 'active') throw conflict('Only active prescriptions can be refilled', '
    INVALID_TRANSITION');
  if (rx.refillRequest.status === 'pending') throw conflict('A refill request is already pending'
    , 'ALREADY_PENDING');
  if (rx.refillsUsed >= rx.refillsAllowed) throw conflict('No refills remain on this prescription
    . Book an appointment for a new one.', 'NO_REFILLS');

  rx.refillRequest = { status: 'pending', requestedAt: new Date(), decidedAt: null };
  await chain(rx, 'REFILL_REQUESTED', req);
  await notificationService.notify(rx.doctorId, {
    type: 'warning', title: 'Refill request', message: `A patient requested a refill of ${rx.
    medication} ${rx.dosage}.`, page: 'prescriptions',
  });
  res.json({ success: true, message: 'Refill requested', data: rx });
}));
```

Listing 11.2: The doctor decides (prescriptions.routes.js)

```
    if (action === 'approve_refill' || action === 'deny_refill') {
      if (rx.refillRequest.status !== 'pending') throw conflict('There is no pending refill
    request', 'INVALID_TRANSITION');
      const approved = action === 'approve_refill';
      rx.refillRequest.status = approved ? 'approved' : 'denied';
      rx.refillRequest.decidedAt = new Date();
      if (approved) rx.refillsUsed += 1;
      await chain(rx, approved ? 'REFILL_APPROVED' : 'REFILL_DENIED', req);
      // ... notify the patient
    } else {
      if (rx.status !== 'active') throw conflict(`The prescription is already ${rx.status}`, '
    INVALID_TRANSITION');
      rx.status = action === 'complete' ? 'completed' : 'cancelled';
      await chain(rx, action === 'complete' ? 'COMPLETE' : 'CANCEL', req);
    }
```

The doctor can only act on prescriptions they wrote (the query includes `doctorId`). All rule violations return 409 with a precise message, which the dashboard shows as-is. Every change is chained in the ledger, so the dispensing history of a controlled drug can be audited.

11.2 Vital signs

Files. src/models/VitalReading.js, src/models/Device.js, src/modules/vitals/vitals.routes.js.

Listing 11.3: Normal ranges and the abnormal check (src/models/VitalReading.js)

```
const VITAL_RANGES = {
  heartRate: { min: 50, max: 110, unit: 'bpm', label: 'Heart rate' },
  systolic: { min: 90, max: 140, unit: 'mmHg', label: 'Systolic BP' },
  diastolic: { min: 60, max: 90, unit: 'mmHg', label: 'Diastolic BP' },
  spo2: { min: 94, max: 100, unit: '%', label: 'Blood oxygen' },
  temperature: { min: 36.1, max: 37.8, unit: 'C', label: 'Temperature' },
  glucose: { min: 70, max: 180, unit: 'mg/dL', label: 'Blood glucose' },
};

/** Returns the names of the fields in `reading` that fall outside VITAL_RANGES. */
function findAbnormal(reading) {
  return Object.entries(VITAL_RANGES)
    .filter(([field, range]) => reading[field] !== null && reading[field] !== undefined
      && (reading[field] < range.min || reading[field] > range.max))
    .map(([field]) => field);
}
```

(The temperature unit in the source is the degree sign, °C.) The ranges are sent to the frontend with every vitals response, so the charts and tiles colour values with the same thresholds the server uses for alerts.

Listing 11.4: Saving a reading and raising alerts (vitals.routes.js)

```
async function saveReading(patientId, values, source, deviceId = null) {
  const reading = await VitalReading.create({
    patientId, deviceId, source, ...values, abnormal: findAbnormal(values), recordedAt: values.
    recordedAt || new Date(),
  });

  if (reading.abnormal.length) {
    const details = reading.abnormal.map((f) => `${VITAL_RANGES[f].label} ${reading[f]} ${
    VITAL_RANGES[f].unit}`).join(', ');
    const patient = await User.findById(patientId).select('fullName');
    const doctors = await AccessGrant.find({ patientId, status: 'active', expiresAt: { $gt: new
    Date() } }).distinct('doctorId');
    await notificationService.notify(patientId, {
      type: 'danger', title: 'Abnormal vital sign', message: `Out of normal range: ${details}.`,
    page: 'metrics',
    });
    await notificationService.notify(doctors, {
      type: 'danger', title: `Vitals alert: ${patient.fullName}`, message: `Out of normal range:
    ${details}.`, page: 'iot',
    });
  }
  return reading;
}
```

Alerts go to the patient and to every doctor who *currently* has consent: the consent model decides who is told, not just who can look.

11.3 IoT devices

11.3.1 Registration and keys

An admin registers a device (serial number, type, patient). The server generates a random API key, stores only its SHA-256, and shows the key once:

Listing 11.5: Device model (src/models/Device.js)

```
const deviceSchema = new mongoose.Schema({
  serialNumber: { type: String, required: true, unique: true, trim: true },
  name: { type: String, required: true, trim: true },
  type: { type: String, enum: ['heart_monitor', 'bp_monitor', 'pulse_oximeter', 'thermometer', '
    multi_parameter'], required: true },
  patientId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  apiKeyHash: { type: String, required: true, unique: true },
  status: { type: String, enum: ['active', 'inactive'], default: 'active' },
  lastSeenAt: { type: Date, default: null },
  registeredBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
}, { timestamps: true });
```

11.3.2 Ingestion

Listing 11.6: POST /api/devices/readings (vitals.routes.js)

```
router.post('/devices/readings', validate({ body: readingSchema }), asyncHandler(async (req, res)
    => {
  const key = req.get('x-device-key');
  if (!key) throw unauthorized('Device key required', 'NO_DEVICE_KEY');
  const device = await Device.findOne({ apiKeyHash: crypto.createHash('sha256').update(key).
    digest('hex') });
  if (!device || device.status !== 'active') throw unauthorized('Unknown or inactive device', '
    INVALID_DEVICE');

  const reading = await saveReading(device.patientId, req.body, 'device', device._id);
  device.lastSeenAt = new Date();
  await device.save();
  res.status(201).json({ success: true, data: { readingId: reading._id, abnormal: reading.
    abnormal } });
}));
```

How it works

A device is not a user: it has no password, no OTP and no JWT. It proves its identity with its key, and the key's hash identifies the device and therefore the patient. A device cannot choose which patient a reading belongs to, so a stolen key can only add readings for one patient, and the admin can deactivate the device at once. The Joi schema rejects physiologically impossible values (for example a heart rate of 400).

11.3.3 The doctor's live monitor

`GET /api/vitals/monitor` returns, for every patient who currently allows this doctor in, the latest reading and the patient's devices with their `lastSeenAt`. The doctor's *IoT Monitoring* page polls it, so readings from devices (or from the simulator) appear within seconds.

11.3.4 The simulator

scripts/iot-simulator.js stands in for real hardware. It reads the seeded device keys from scripts/.demo-devices.json (git-ignored) and posts readings to the real ingestion endpoint:

Listing 11.7: Simulated readings (scripts/iot-simulator.js)

```
function readingFor(device) {
  if (spike) {
    return device.type === 'pulse_oximeter'
      ? { heartRate: Math.round(vary(128, 4)), spo2: Math.round(vary(89, 1)) }
      : { heartRate: Math.round(vary(124, 4)), systolic: Math.round(vary(168, 4)), diastolic:
    Math.round(vary(104, 3)), spo2: 96, temperature: vary(37.2, 0.2) };
  }
  if (device.type === 'pulse_oximeter') return { heartRate: Math.round(vary(80, 6)), spo2: Math.
    round(vary(97, 1)) };
  return {
    heartRate: Math.round(vary(74, 6)), systolic: Math.round(vary(126, 5)), diastolic: Math.round
    (vary(81, 3)),
    spo2: Math.round(vary(97.5, 1)), temperature: vary(36.8, 0.2),
  };
}
```

With `-spike`, the first reading from each device is clearly abnormal (blood pressure around 168/104, SpO_2 around 89%) and later ones are normal again: a single burst that demonstrates the alert path end to end.

CHAPTER 12

Billing and Insurance Claims

Files. src/models/Bill.js, src/models/Claim.js, src/modules/billing/, src/modules/claims/claims.routes.js.

12.1 Bills

A bill is never typed in by hand. It is raised automatically when a doctor marks an appointment *completed*, at the doctor's consultation fee:

Listing 12.1: createBillForAppointment (src/modules/billing/billing.service.js)

```
/** Human-readable unique number, e.g. "B-20261027-3FA9C1". */
function makeNumber(prefix) {
  const d = new Date();
  const date = `${d.getFullYear()}${String(d.getMonth() + 1).padStart(2, '0')}${String(d.getDate
    ()).padStart(2, '0')}`;
  return `${prefix}-${date}-${crypto.randomBytes(3).toString('hex').toUpperCase()}`;
}

/** Raises the consultation bill when a doctor completes an appointment. */
async function createBillForAppointment(appointment, doctor, actor) {
  const bill = await Bill.create({
    billNumber: makeNumber('B'),
    patientId: appointment.patientId,
    doctorId: appointment.doctorId,
    appointmentId: appointment._id,
    description: `Consultation with Dr. ${doctor.fullName} (${doctor.doctorProfile.specialization
     || 'General'})`,
    amount: doctor.doctorProfile.consultationFee,
  });
  await ledgerService.recordEvent({ entityType: 'bill', entity: bill, action: 'CREATE', actor });
  // ... notify the patient: "A bill of Rs. ... was raised"
  return bill;
}
```

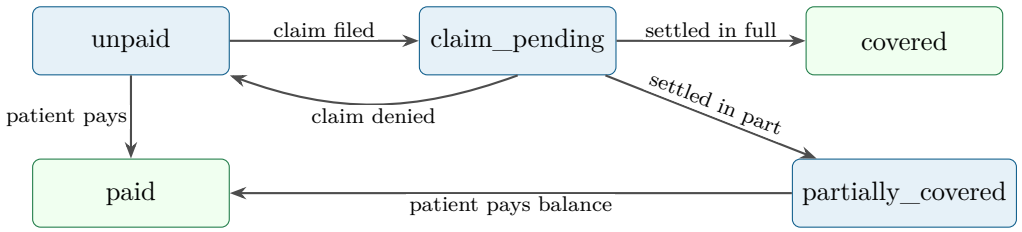


Figure 12.1: Bill states

`POST /api/bills/:id/pay` records that the patient paid the outstanding balance (amount minus what insurance covered). There is no payment gateway: the endpoint records the payment and chains it in the ledger (`PAYMENT_RECORDED`). Connecting a real gateway is listed as future work.

12.2 Claims

12.2.1 Filing a claim

Listing 12.2: POST /api/claims (claims.routes.js)

```
    const patient = await User.findById(req.user.userId);
    const { insuranceProvider, policyNumber } = patient.patientProfile;
    if (!insuranceProvider || !policyNumber) {
      throw badRequest('Add your insurance provider and policy number in Settings before filing a
     claim');
    }
    const bill = await Bill.findOne({ _id: req.body.billId, patientId: patient._id });
    if (!bill) throw notFound('Bill not found');
    if (bill.status !== 'unpaid') throw conflict(`This bill is ${bill.status.replace('_', ' ')}
    and cannot be claimed`, 'INVALID_TRANSITION');

    const claim = new Claim({
      claimNumber: makeNumber('CLM'),
      patientId: patient._id,
      billId: bill._id,
      insuranceProvider,
      policyNumber,
      amount: bill.amount,
      description: req.body.description || bill.description,
      history: [{ status: 'submitted', by: req.user.userId, byRole: 'patient', note: 'Claim
    submitted' }],
    });
    await chain(claim, 'SUBMIT', req);
    bill.status = 'claim_pending';
    await bill.save();
    // ... ledger entry for the bill, notify the insurer's agents
```

How it works

A claim always refers to a real bill of the same patient, for the bill's exact amount; the patient cannot invent an amount. The provider and policy number come from the patient's profile. Only unpaid bills can be claimed, and filing moves the bill to `claim_pending` so it cannot be claimed twice or paid while under review. Only agents of that insurer are notified.

12.2.2 The claim state machine

Listing 12.3: Allowed transitions (src/models/Claim.js)

```
const CLAIM_TRANSITIONS = {
  submitted: ['under_review'],
  under_review: ['approved', 'denied'],
  approved: ['settled'],
  denied: [],
  settled: [],
};
```

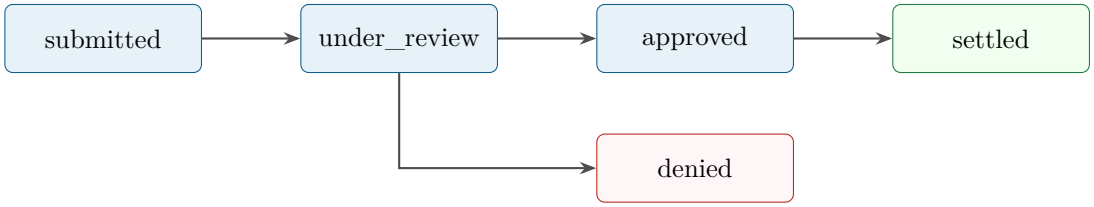


Figure 12.2: Claim states

Listing 12.4: Processing a claim (`claims.routes.js`)

```
const { status, approvedAmount, note } = req.body;
if (!CLAIM_TRANSITIONS[claim.status].includes(status)) {
  throw conflict(`A claim cannot move from "${claim.status}" to "${status}"`, '
INVALID_TRANSITION');
}
if (status === 'approved') {
  if (approvedAmount === undefined) throw badRequest('approvedAmount is required to approve a
 claim');
  if (approvedAmount > claim.amount) throw badRequest('approvedAmount cannot exceed the
claimed amount');
  claim.approvedAmount = approvedAmount;
}
if (status === 'denied' && !note) throw badRequest('Give a reason when denying a claim');

claim.status = status;
claim.agentId = req.user.userId;
if (note) claim.decisionNote = note;
claim.history.push({ status, by: req.user.userId, byRole: 'insurance_agent', note });

const bill = await Bill.findById(claim.billId);
if (status === 'settled') {
  claim.settledAt = new Date();
  claim.settlementReference = `STL-${crypto.randomBytes(4).toString('hex').toUpperCase()}`;
  bill.coveredAmount = claim.approvedAmount;
  bill.status = claim.approvedAmount >= bill.amount ? 'covered' : 'partially_covered';
} else if (status === 'denied') {
  bill.status = 'unpaid'; // the patient pays it themselves
}
await chain(claim, status.toUpperCase(), req, note ? { note } : undefined);
```

How it works

- The transition table is the single source of truth: anything not listed (for example `submitted` → `settled`) is a 409.
- Approval needs an amount no larger than the claim; denial needs a reason, which the patient sees.
- Settling generates a settlement reference and updates the bill, so the patient's Billing page shows what insurance paid and what is left.
- Agents only see and act on claims of their own insurer (`agentScope`); a claim of another insurer returns 404 or 403.
- Every step is appended to the claim's `history` *and* chained in the ledger; the claim detail view shows both.

12.2.3 Settlements and reports

`GET /api/claims/settlements` lists settled claims. `GET /api/claims/reports` aggregates, with MongoDB's aggregation pipeline, claims by status, claimed versus settled amounts per month, the approval rate and the average time from submission to settlement; the agent's *Reports* page charts these numbers. All figures are computed from live data at request time.

CHAPTER 13

Administration, Notifications and Settings

Files. src/modules/admin/admin.routes.js, src/services/notificationService.js, src/modules/notifications/notifications.routes.js, src/models/HospitalSettings.js. Every admin route is behind `router.use(authenticate, requireRole('admin'))`.

13.1 Credential verification

Doctors and insurance agents who register themselves start *pending*. `GET /api/admin/doctors/pending-verification` lists them (with licence number or company), and the admin decides:

Listing 13.1: Approving or rejecting (admin.routes.js)

```
const user = await User.findOne({ _id: req.params.id, role: { $in: User.ROLES_NEEDING_APPROVAL
  } });
if (!user) throw notFound('Account not found');
if (user.approvalStatus !== 'pending') throw conflict(`This account is already ${user.
  approvalStatus}`, 'ALREADY_DECIDED');

const approved = req.body.decision === 'approve';
user.approvalStatus = approved ? 'approved' : 'rejected';
user.approvalNote = req.body.note;
await user.save();
await invalidateSession(user._id); // the new status applies on their next request
await ledgerService.append({
  chain: 'records', entityType: 'user', entityId: user._id, action: approved ? '
  CREDENTIALS_VERIFIED' : 'CREDENTIALS_REJECTED',
  actorId: req.user.userId, actorRole: 'admin', payload: { approvalStatus: user.approvalStatus,
   licenseNumber: user.doctorProfile?.licenseNumber },
});
// ... in-app notification + email to the user
```

The decision takes effect on the doctor's very next request, because the session cache is invalidated (Chapter 7); there is no need for them to log out. The verification, including which licence number was verified and by whom, is permanently recorded in the ledger.

13.2 User management

- `GET /api/admin/users?role=&status=&q=` searches by name or email and filters by role and active/pending state.
- `POST /api/admin/users` creates staff accounts (admin, doctor, agent) that are approved immediately. This is the only way, besides the seed, to create an admin.
- `PATCH /api/admin/users/:id { isActive }` activates or deactivates. Deactivation revokes every refresh token and invalidates the session cache, so the user is out on their next click. An admin cannot deactivate themselves (so the hospital cannot lock itself out by mistake).

13.3 Live statistics

`GET /api/admin/stats` computes every number on the admin overview at request time, in one round of parallel queries:

Listing 13.2: Parallel aggregation (`admin.routes.js`, abridged)

```
const [usersByRole, pendingVerification, appointmentsToday, appointmentsByStatus, weekly,
  records, claimsByStatus, revenue,
  devices, readings24h, activeConsents, ledger, mongoPing, redisPing] = await Promise.all([
  User.aggregate([{ $group: { _id: '$role', total: { $sum: 1 }, active: { $sum: { $cond: ['
  $isActive', 1, 0] } } } }]),
  User.countDocuments({ approvalStatus: 'pending' }),
  // ... appointments today, by status, per day for 7 days
  MedicalRecord.countDocuments(),
  Claim.aggregate([{ $group: { _id: '$status', count: { $sum: 1 }, amount: { $sum: '$amount' }
  } }]),
  Bill.aggregate([{ $group: { _id: null, billed: { $sum: '$amount' }, paid: { $sum: '
  $paidAmount' }, covered: { $sum: '$coveredAmount' } } }]),
  // ... devices, readings in 24 h, active consents
  ledgerService.stats(),
  ping(() => mongoose.connection.db.admin().ping()),
  ping(() => redisClient.ping()),
]);
```

Because nothing is cached or hardcoded, registering a new user changes the user count on the next refresh; an integration test checks exactly that. The `system` section reports uptime, memory, load and the latency of a MongoDB and Redis ping, which is useful during a demo to show the infrastructure is live.

13.4 Hospital settings

One settings document (`HospitalSettings.getSingleton()`) holds the hospital name and contacts, the booking window, the default consent duration, the list of insurers and the departments. The public part is served without login by `GET /api/settings/public`, so the registration form's insurer and department dropdowns come from the database too. Saving settings is a ledger entry.

13.5 Access audit

`GET /api/admin/audit` reads the `access` chain newest first, filtered by decision (allow/deny) or reason, with the actor's name resolved. It also returns two counters for the page header: denials in the last 24 hours and the total number of break-glass accesses. This is the page an auditor uses to answer "who tried to open what, and were they allowed?".

13.6 IoT device registry

`GET/POST/PATCH /api/admin/devices` lists devices with their patient and last-seen time, registers new ones (returning the API key once, Chapter 11) and activates or deactivates them.

13.7 Notifications

Listing 13.3: notify and notifyRole (src/services/notificationService.js)

```
/**
 * Creates in-app notifications. Failures are logged and swallowed: a missed
 * notification must never roll back the clinical or financial action that
 * caused it.
 */
async function notify(userIds, { type = 'info', title, message, page = '' }) {
  const ids = [].concat(userIds).filter(Boolean).map(String);
  if (!ids.length) return [];
  try {
    return await Notification.insertMany(ids.map((userId) => ({ userId, type, title, message,
    page })));
  } catch (error) {
    logger.error(`Notification failed: ${error.message}`);
    return [];
  }
}

async function notifyRole(role, payload) {
  const users = await User.find({ role, isActive: true }).select('_id').lean();
  return notify(users.map((u) => u._id), payload);
}
```

How it works

`page` names the dashboard page the notification is about; clicking a notification in the browser navigates there. `notify` accepts one id or a list, so "tell every doctor with consent" is one call. Notifications are deliberately best-effort, unlike ledger writes: if the ledger write fails, the request fails; if a notification fails, it is only logged. The dashboards poll `GET /api/notifications` every 20 seconds to update the bell and sidebar badges; `PATCH /:id/read` and `POST /read-all` mark them read.

Table 13.1: Who is notified of what

Event	Recipients
Doctor/agent registers	All admins
Appointment booked / rescheduled	Doctor
Appointment confirmed, completed, cancelled	Patient (or the other party)
Access requested	Patient
Access granted, approved, denied, revoked	Doctor
Break-glass access	Patient and all admins
New record, prescription or bill	Patient
Refill requested / decided	Doctor / patient
Abnormal vital sign	Patient and every doctor with active consent
Claim filed	Agents of that insurer
Claim status changes	Patient
Account verified / rejected	The user (in-app and email)

CHAPTER 14

Frontend

14.1 Structure

The frontend is plain HTML, CSS and JavaScript with no build step. The API server serves it from `frontend/`.

Table 14.1: Frontend files

File	Role
`homepage.html`, `homepagescript.js`	Public landing page; live counts from `GET /api/public/stats`
`login.html`, `loginscript.js`	Sign in (password, then OTP), registration for three roles
`patient.html`, `doctor.html`, `admin.html`, `insurance.html`	Empty shells: they load the shared scripts and one role script
`js/api.js`	The only code that talks to the server (tokens, refresh, errors)
`js/ui.js`	Escaping, formatting, loading/empty/error states, tables, modals, toasts
`js/portal.js`	Dashboard shell: session restore, sidebar, routing, notifications, profile
`js/patient.js`, `doctor.js`, `admin.js`, `insurance.js`	One function per page of that role's dashboard
`css/portal.css`	Shared design system (light and dark), responsive layout

A dashboard shell is only a few lines: it loads two pinned CDN libraries (icons and charts) and the four scripts, all with `defer`.

Listing 14.1: Script tags of `frontend/patient.html`

```
<script src="https://unpkg.com/lucide@0.460.0/dist/umd/lucide.min.js" defer></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js" defer></script>
<script src="js/api.js" defer></script>
<script src="js/ui.js" defer></script>
<script src="js/portal.js" defer></script>
<script src="js/patient.js" defer></script>
```

14.2 The API client: `js/api.js`

Listing 14.2: request with transparent refresh (frontend/js/api.js)

```
async function request(method, path, { body, query, form, retry = true } = {}) {
  // ... build the URL from path + query
  const headers = {};
  if (accessToken) headers.Authorization = `Bearer ${accessToken}`;
  if (breakGlassReason) headers['x-break-glass-reason'] = breakGlassReason;
  // ... JSON body or FormData

  let response;
  try {
    response = await fetch(url, { method, headers, body: payload, credentials: 'same-origin' })
  ;
  } catch (networkError) {
    throw new ApiError(0, 'Cannot reach the MediChain server. Check that it is running.', null)
  ;
  }

  const expired = response.status === 401 && retry && !path.startsWith('/api/auth/');
  if (expired) {
    try {
      await refreshSession();
    } catch (refreshError) {
      accessToken = null;
      throw new ApiError(401, 'Your session has ended. Please log in again.', { code: '
  SESSION_ENDED' });
    }
    return request(method, path, { body, query, form, retry: false });
  }
  return parse(response);
}
```

Listing 14.3: One shared refresh (frontend/js/api.js)

```
function refreshSession() {
  if (!refreshing) {
    refreshing = fetch('/api/auth/refresh-token', { method: 'POST', credentials: 'same-origin'
  })
      .then(parse)
      .then((body) => {
        accessToken = body.data.token;
        return body.data.user;
      })
      .finally(() => { refreshing = null; });
  }
  return refreshing;
}
```

How it works

- `accessToken` is a variable inside the module's closure: never in `localStorage`, never reachable from other scripts.
- When any call gets a 401 (access token expired after 15 minutes), the client silently refreshes once and retries the call. The user never sees it.
- A dashboard page often makes five calls at once; if all five get 401, `refreshing` makes them share *one* refresh request. This matters because refresh tokens rotate: five parallel refreshes would invalidate each other.
- A network failure becomes `ApiError` with status 0, which the screens show as "Server unreachable".

- While emergency access is switched on, every request carries the `x-break-glass-reason` header.

14.3 The dashboard shell: `js/portal.js`

14.3.1 Start-up

Listing 14.4: Portal.start (`frontend/js/portal.js`)

```
async function start(config) {
  state.config = config;
  applyTheme(savedTheme());
  let user;
  try {
    user = await api.refreshSession();
  } catch (error) {
    if (error.status === 0) {
      document.getElementById('app').innerHTML = '<div class="boot-screen"><div class="state
  state-error"><h4>Server unreachable</h4><div>Start the MediChain API and reload this page.</
  div></div></div>';
      return;
    }
    sendToLogin();
    return;
  }
  if (user.role !== config.role) {
    location.replace(DASHBOARDS[user.role]);
    return;
  }
  state.user = user;
  document.getElementById('app').addEventListener('click', onClick); // one delegated listener
  for the whole app
  renderLayout();
  renderBanners();
  window.addEventListener('hashchange', () => showPage());
  await showPage();
  refreshBadges();
  state.pollTimer = setInterval(refreshBadges, 20000);
}
```

Opening `doctor.html` while logged in as a patient sends you to `patient.html`. This is a convenience only: the real protection is the server's role check on every endpoint.

14.3.2 Rendering a page: real loading, empty and error states

Listing 14.5: showPage (`frontend/js/portal.js`)

```
async function showPage(pageId = currentPageId()) {
  const token = ++state.renderToken;
  state.page = pageId;
  state.handlers = {};
  state.charts.forEach((c) => c.destroy());
  state.charts = [];
  // ... highlight the nav item, set the title
  const container = document.getElementById('page-container');
  UI.loading(container);
  const render = state.config.pages[pageId] || SHARED_PAGES[pageId];
  try {
    await render(ctx(container, token));
```

```
  } catch (error) {
    if (token !== state.renderToken) return;
    if (error.code === 'SESSION_ENDED') return sendToLogin();
    UI.errorState(container, error, () => showPage(pageId));
  }
  if (token === state.renderToken) refreshIcons();
}
```

How it works

The URL hash (`#appointments`) selects the page. The shell shows a loading state, then calls the role script's page function with a context object `c`. If the function throws (any `ApiError`), the shell replaces the page with an error state and a *Try again* button. Nothing is ever shown from a previous load or a fixture. The `renderToken` solves a race: if the user clicks another page while the first is still loading, the first page's late result is discarded (`c.stale()` becomes true).

Listing 14.6: The error state (frontend/js/ui.js)

```
function errorState(container, error, onRetry) {
  const offline = error && error.status === 0;
  container.innerHTML = `<div class="state state-error" role="alert">
    ${icon(offline ? 'wifi-off' : 'alert-triangle')}
    <h4>${offline ? 'Server unreachable' : 'Could not load this page'}</h4>
    <div>${esc(error && error.message)}</div>
    ${onRetry ? '<button class="btn btn-secondary mt-3" data-retry>Try again</button>' : ''}
  </div>`;
  if (onRetry) container.querySelector('[data-retry]').addEventListener('click', onRetry);
  refreshIcons();
}
```

14.3.3 Escaping: the second line of XSS defence

Listing 14.7: esc (frontend/js/ui.js)

```
function esc(value) {
  return String(value ?? '')
    .replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;').replace(/'/g, '&#39;');
}
```

Pages build HTML with template strings, so *every* value that came from the server goes through `esc` before insertion. A patient named `<img src=x onerror=alert(1)>` is displayed as text. A frontend test checks this function, and the CSP (Section 5) blocks inline scripts even if an escape were forgotten.

14.4 A complete page: the patient's Permissions screen

Every role page follows the same pattern: fetch in parallel, bail out if stale, render, register click handlers.

Listing 14.8: Data loading and handlers (frontend/js/patient.js, abridged)

```
async function permissions(c) {
  const [consents, doctors] = await Promise.all([c.api.get('/api/consents'), c.api.get('/api/
  doctors')]);
  if (c.stale()) return;
```

```
  const requested = consents.data.filter((g) => g.status === 'requested');
  const active = consents.data.filter((g) => g.status === 'active');
  // ... c.container.innerHTML = cards and tables built from these arrays,
  //     with buttons like <button data-action="revoke" data-id="...">

  c.on({
    approve: (el) => UI.modal({ /* choose a duration, then POST /api/consents/:id/approve */ })
  ,
    deny: (el) => c.act(() => c.api.post(`/api/consents/${el.dataset.id}/deny`)),
    revoke: async (el) => {
      if (await UI.confirmDialog('Revoke access', 'The doctor will lose access to your records
  immediately.', { confirmLabel: 'Revoke', danger: true })) {
        await c.act(() => c.api.post(`/api/consents/${el.dataset.id}/revoke`));
      }
    },
    // ... grant
  });
}
```

How it works

Buttons carry `data-action` and `data-id` attributes instead of `onclick` code (which the CSP would block). The shell's single click listener finds the nearest `[data-action]` element and calls the handler the page registered with `c.on`. `c.act` runs an API call, shows a success or error toast with the server's message, reloads the page and refreshes the badge counters. Empty lists render through `UI.table(..., 'No doctor currently has access')`, which shows a real empty state instead of an empty table.

14.5 Login: `loginscript.js`

Listing 14.9: Two-step sign-in (`frontend/loginscript.js`, abridged)

```
async function requestCode(button) {
  setLoading(button, true);
  try {
    const res = await api.post('/api/auth/login/request-otp', { email: pending.email, password:
   pending.password });
    $('verificationContact').textContent = res.data.email;
    $('otpMinutes').textContent = res.data.expiresInMinutes;
    return true;
  } catch (error) {
    const remaining = error.body && error.body.attemptsRemaining;
    setMessage('loginMessage', remaining !== undefined ? `${error.message}. ${remaining}
  attempt(s) left before the account locks.` : error.message);
    return false;
  } finally {
    setLoading(button, false);
  }
}

async function onVerify() {
  const otp = codeInputs().map((i) => i.value).join('');
  if (!/^\d{6}$/.test(otp)) { setMessage('otpMessage', 'Enter all 6 digits of the code.');
  return; }
  // ...
    const res = await api.post('/api/auth/login/verify-otp', { email: pending.email, otp });
    pending.password = '';
    // ... redirect to the dashboard for res.data.user.role
}
```

```
function redirectAfterLogin(role) {
  const next = new URLSearchParams(location.search).get('next');
  // Only follow "next" when it points at this user's own dashboard (prevents open redirects).
  const target = next && next.startsWith(DASHBOARDS[role]) ? next : DASHBOARDS[role];
  location.replace(target);
}
```

The password is kept in memory only between the two steps (so "resend code" works) and cleared as soon as the code is accepted. The six code boxes accept typing, pasting a whole code, and backspace across boxes. The registration form loads its insurer and department lists from `GET /api/settings/public`.

14.6 What each dashboard shows

Table 14.2: Dashboard pages and their endpoints

Page	Content (endpoints)
Patient	
Dashboard	Next appointment, access requests, active medications, latest vitals, unpaid bills (appointments, prescriptions, consents, bills, vitals)
Appointments	Upcoming and history; book with a live slot grid; reschedule; cancel
Medical Records	Records with attachments, upload a document, download, ledger history, Verify integrity
Medications	Active and past prescriptions; request refill
Health Metrics	Charts of the last 7 days of vitals, abnormal-reading warning, registered devices, add a reading
Permissions	Pending requests, active grants, history; approve, deny, grant, revoke
Billing & Claims	Bills, record payment, file claim, claim timeline
Doctor	
Dashboard	Today's schedule, pending confirmations, refill queue, consent counts
Appointments	Today / Upcoming / Past; confirm, complete (with notes), no-show, cancel
My Patients	Patients with consent status; find by email; request access; patient chart with records, vitals, prescriptions, new record, upload, prescribe
Prescriptions	All prescriptions; approve or deny refills; complete or cancel
IoT Monitoring	Live tiles of each consenting patient's latest vitals, refreshed every 5 s
Emergency access	Header button that switches break-glass on with a reason
Admin	
Dashboard / Reports	Live statistics and charts (`/api/admin/stats`)

Page	**Content (endpoints)**
Verification	Pending doctors and agents; approve or reject
Users	Search, filter, activate/deactivate, create staff
IoT Devices	Registry; register (key shown once); activate/deactivate
Hospital Settings	Name, contacts, booking window, consent duration, insurers, departments
Ledger	Chain statistics, browse blocks, Verify ledger
Access Control	The access chain: allow/deny decisions, break-glass uses
Insurance agent	
Dashboard	Queue size, amounts, approval rate
Claims Queue	Filter by status; process a claim through the state machine
Settlements, Reports	Settled claims; monthly and status aggregates with charts
All roles	
Notifications	List, mark read, jump to the related page
Profile & Security	Edit profile (and availability for doctors), change password, current trust score with its reasons

CHAPTER 15

Reference: Data Model and API

15.1 MongoDB collections

Table 15.1: Collections

Collection	Holds	Key indexes / rules
`users`	All accounts and role profiles, approval, lockout	unique `email`; `(role, approvalStatus)`
`appointments`	Bookings with status, notes, cancellation	unique `(doctorId, startTime)` where `holdsSlot`
`accessgrants`	Patient consents to doctors	`(patientId, doctorId, status)`
`medicalrecords`	Clinical records, versions, attachment metadata	`patientId`
`uploads.files`, `uploads.chunks`	GridFS file storage	managed by GridFS
`prescriptions`	Medications, refills	`patientId`, `doctorId`
`vitalreadings`	Device and manual readings, abnormal flags	`(patientId, recordedAt)`
`devices`	IoT devices, hashed API keys	unique `serialNumber`, `apiKeyHash`
`bills`	Consultation bills and payment/coverage	unique `billNumber`
`claims`	Insurance claims with history	unique `claimNumber`; `insuranceProvider`
`notifications`	In-app notifications	by user
`hospitalsettings`	The single settings document	unique `key`
`ledgerentries`	Both hash chains	unique `(chain, seq)`

Redis keys are listed in Table 6.2 (Section 6.7).

15.2 Response format and error codes

Success: `{ "success": true, "data": ..., "message"?: ..., "meta"?: ... }`. Failure: `{ "success": false, "message": ..., "code"?: ..., "errors"?: [...] }`.

Table 15.2: HTTP status codes

Code	Meaning in MediChain
400	Validation failed (`errors` lists every problem), malformed JSON, invalid id
401	Not logged in, token expired or revoked, wrong password or code
403	Wrong role, approval pending, trust score too low, no consent, account locked
404	Not found, or not yours (ids of other users' items are not confirmed)
409	Conflict: slot taken, invalid state transition, duplicate request
413	Body or file too large
429	Rate limit or OTP attempt limit reached
503	Redis (security state) unavailable: the request fails closed

15.3 Endpoints

All paths start with `/api`. "Auth" means any logged-in user; roles are enforced by `requireRole`. "HS" marks routes that require the high-sensitivity trust score (70); "BG" marks routes that accept break-glass.

Table 15.3: API endpoints

Method	Path	Who	Purpose
Auth			
POST	`/auth/register`	public	Create patient, doctor or agent account
POST	`/auth/login/request-otp`	public	Check password, email a code
POST	`/auth/login/verify-otp`	public	Exchange the code for tokens
POST	`/auth/refresh-token`	cookie	Rotate refresh token, new access token
POST	`/auth/logout`	Auth	Revoke both tokens
POST	`/auth/change-password`	Auth	Change password, sign out everywhere
Public, profile and directory			
GET	`/settings/public`	public	Hospital name, insurers, departments
GET	`/public/stats`	public	Anonymous counts for the homepage
GET	`/users/me`	Auth	Own account and current trust score
PATCH	`/users/me`	Auth	Edit own profile (own role's section only)
GET	`/doctors`	Auth	Verified doctors
GET	`/doctors/:id/available-slots`	Auth	Slots of a day with availability
GET	`/patients`	doctor	Doctor's patients with consent status

Method	Path	Who	Purpose
GET	`/patients/search?email=`	doctor	Find a patient to request access
Appointments			
POST	`/appointments`	patient	Book a slot
GET	`/appointments`	Auth	Own appointments (all for admin)
GET	`/appointments/:id`	party/admin	One appointment
PATCH	`/appointments/:id`	doctor/patient	Status + notes (doctor), reschedule (patient)
DELETE	`/appointments/:id`	party/admin	Cancel, freeing the slot
Consent			
GET	`/consents`	patient, doctor	Own grants and requests
POST	`/consents/request`	doctor	Ask a patient for access
POST	`/consents/grant`	patient	Grant a doctor access directly
POST	`/consents/:id/approve`	patient	Approve a request with a duration
POST	`/consents/:id/deny`	patient	Deny a request
POST	`/consents/:id/revoke`	patient	End access immediately
Records and files			
POST	`/records`	patient, doctor (HS)	Create record / own upload entry
GET	`/records?patientId=`	patient, doctor (HS, BG)	List a patient's records
GET	`/records/:id`	patient, doctor (HS, BG)	One record
PATCH	`/records/:id`	author doctor (HS)	Amend, new version
GET	`/records/:id/ledger-history`	patient, doctor (HS, BG)	Ledger trail of the record
GET	`/records/:id/verify`	patient, doctor (HS, BG)	Integrity check of content and files
POST	`/records/:id/files`	patient, doctor (HS)	Upload an attachment
GET	`/files/:fileId`	patient, doctor (HS, BG)	Download, with `X-Content-SHA256`
Prescriptions			
POST	`/prescriptions`	doctor (HS)	Prescribe (needs consent)
GET	`/prescriptions`	patient, doctor	Own / a patient's prescriptions
POST	`/prescriptions/:id/refill-request`	patient	Ask for a refill

Method	Path	Who	Purpose
PATCH	`/prescriptions/:id`	author doctor	Approve/deny refill, complete, cancel
Vitals and IoT			
POST	`/devices/readings`	device key	Ingest a device reading
POST	`/vitals`	patient	Add a manual reading
GET	`/vitals`	patient, doctor (BG)	History with normal ranges
GET	`/vitals/monitor`	doctor	Latest reading of every consenting patient
GET	`/devices/mine`	patient	Own devices
Billing and claims			
GET	`/bills`	patient, admin	Bills
POST	`/bills/:id/pay`	patient	Record payment of the balance
POST	`/claims`	patient	File a claim on an unpaid bill
GET	`/claims`	patient, agent, admin	Own claims / insurer's queue / all
GET	`/claims/settlements`	agent, admin	Settled claims
GET	`/claims/reports`	agent, admin	Aggregated figures
GET	`/claims/:id`	patient, agent, admin	One claim with ledger history
PATCH	`/claims/:id`	agent	Move through the state machine
Notifications			
GET	`/notifications`	Auth	Own notifications and unread count
PATCH	`/notifications/:id/read`	Auth	Mark one read
POST	`/notifications/read-all`	Auth	Mark all read
Admin (all: admin role)			
GET	`/admin/users`	admin	Search and filter users
POST	`/admin/users`	admin	Create a staff account
PATCH	`/admin/users/:id`	admin	Activate / deactivate
GET	`/admin/doctors/pending-verification`	admin	Doctors and agents awaiting approval
PATCH	`/admin/doctors/:id/verify`	admin	Approve or reject
GET	`/admin/stats`	admin	Live statistics
GET, PATCH	`/admin/settings`	admin	Read / change hospital settings
GET	`/admin/audit`	admin	Access chain (zero-trust decisions)
GET, POST	`/admin/devices`	admin	List / register IoT devices

Method	Path	Who	Purpose
PATCH	`/admin/devices/:id`	admin	Activate / deactivate a device
Ledger (all: admin role)			
GET	`/ledger/stats`	admin	Entries and head hash per chain
GET	`/ledger/entries`	admin	Browse blocks, newest first
GET	`/ledger/verify`	admin	Full integrity audit

15.4 Glossary

Break-glass
Audited emergency access that bypasses consent.

Canonical JSON
JSON with sorted keys and fixed formats, so equal data always gives an equal hash.

Chain head
The newest entry of a hash chain; its hash fingerprints the whole history.

Consent / grant
A patient's permission for one doctor, with an expiry.

GridFS
MongoDB's way of storing large files in chunks.

JWT
JSON Web Token: a signed, self-contained token carrying the user id, role and expiry.

jti The unique id inside a JWT, used to revoke that exact token.

OTP
One-time password: the 6-digit emailed login code.

RBAC / ABAC
Role-based and attribute-based access control.

Trust score
0–100 estimate of how likely a request comes from the account's real owner.

CHAPTER 16

Testing and Deployment

16.1 Backend tests

Tools. Jest runs the tests; Supertest sends real HTTP requests to the Express app in-process. Tests run against *real* MongoDB and Redis, not mocks, because most of the guarantees under test (unique indexes, TTLs, concurrent appends) live in those systems.

16.1.1 Isolation

Listing 16.1: Test environment (tests/env.js)

```
// Runs before any module is loaded, so config.js sees the test settings.
// Tests use their own MongoDB database and Redis DB index, never the dev data.
process.env.NODE_ENV = 'test';
// Each parallel Jest worker gets its own database and Redis DB index.
const worker = Number(process.env.JEST_WORKER_ID || 1);
process.env.MONGODB_URI = `${process.env.TEST_MONGODB_BASE || 'mongodb://127.0.0.1:27017/
    medichain_test'}_${worker}`;
process.env.REDIS_DB = String(15 - ((worker - 1) % 8));
process.env.JWT_SECRET = 'test-access-secret-0123456789abcdef';
process.env.JWT_REFRESH_SECRET = 'test-refresh-secret-fedcba9876543210';
process.env.EMAIL_TRANSPORT = 'json';
```

Running the tests never touches the development database or sends an email: they use `medichain_test_<n>` databases, Redis DB 8–15, and the `json` mail transport, which renders messages in memory.

16.1.2 Helpers that use the real login flow

Listing 16.2: Logging in inside a test (tests/helpers.js)

```
async function login(user, password = PASSWORD) {
  let otp;
  const spy = jest.spyOn(emailService, 'sendOTP').mockImplementation(async (email, code) => { otp
     = code; });
  const step1 = await request(app).post('/api/auth/login/request-otp').set('User-Agent', 'jest').
    send({ email: user.email, password });
  spy.mockRestore();
  if (step1.status !== 200) throw new Error(`request-otp failed: ${step1.status} ${JSON.stringify
    (step1.body)}`);

  const step2 = await request(app).post('/api/auth/login/verify-otp').set('User-Agent', 'jest').
    send({ email: user.email, otp });
  if (step2.status !== 200) throw new Error(`verify-otp failed: ${step2.status} ${JSON.stringify(
    step2.body)}`);
  const cookie = step2.headers['set-cookie'].find((c) => c.startsWith('refreshToken='));
  return { token: step2.body.data.token, cookie, user: step2.body.data.user };
}
```

There is no test-only login shortcut in the application. The helper goes through both real steps and only intercepts the email to read the code, so every test that logs in also exercises password checking, OTP hashing in Redis and token issuance. `resetDatabase` empties every collection and

Redis before each test and re-creates indexes, so the uniqueness guarantees are present in every test.

16.1.3 What is tested

Table 16.1: Backend test suites

File	Covers
unit/trustService.test.js	Score penalties, policy order, break-glass rules, time windows
unit/ledgerService.test.js	Chaining, concurrent appends, each kind of tampering
unit/slots-hash-config.test.js	Slot generation, canonical JSON, config validation
integration/auth.test.js	Register, both login steps, wrong codes, lockout, rotation, logout, password change, Redis outage (503)
integration/appointments.test.js	Slots, booking, racing double-booking, transitions, cancellation
integration/records.test.js	Consent gate, break-glass, versions, files and hashes, tamper detection
integration/clinical-financial.test.js	Prescriptions, vitals and alerts, devices, billing, claims state machine, admin
integration/edge-cases.test.js	Error shapes, CSP and static files, rescheduling, directory, ledger browsing, email rendering

Listing 16.3: The double-booking race (tests/integration/appointments.test.js)

```
test('the backend rejects double-booking, even when two requests race', async () => {
  const { patient, other, doctorUser } = await world();
  const slot = futureSlot();
  const results = await Promise.all([book(patient, doctorUser._id, slot), book(other,
  doctorUser._id, slot)]);
  expect(results.map((r) => r.status).sort()).toEqual([201, 409]);
  expect(results.find((r) => r.status === 409).body.code).toBe('SLOT_TAKEN');
  expect(await Appointment.countDocuments({ holdsSlot: true })).toBe(1);
});
```

Result (October 2026): 105 tests in 8 suites pass; coverage is about 93% of lines and 90% of functions. Run with `npm test` or `npm run test:coverage` in backend/services/api.

16.2 Frontend tests

frontend/tests uses Jest with the jsdom environment and `@testing-library/dom`. api.test.js replaces `fetch` with a mock server and checks the Bearer header, the transparent refresh, that concurrent 401s share one refresh, the `SESSION_ENDED` path, error mapping, network failures and the break-glass header. ui.test.js checks escaping, formatters, the loading, empty and error states, the retry button, modal forms and toasts. 20 tests pass (`cd frontend && npm test`).

16.3 Docker

Listing 16.4: Dockerfile

```
FROM node:20-alpine

WORKDIR /app/backend/services/api
COPY backend/services/api/package.json backend/services/api/package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY backend/services/api/src ./src
COPY backend/services/api/scripts ./scripts
COPY frontend /app/frontend

# Writable by the unprivileged user: winston logs and the seed's demo-device key file.
RUN mkdir -p /app/backend/logs && chown -R node:node /app/backend/logs ./scripts

ENV NODE_ENV=production PORT=8001
EXPOSE 8001
USER node
HEALTHCHECK --interval=15s --timeout=3s --start-period=20s CMD wget -qO- http://127.0.0.1:8001/
    health || exit 1
CMD ["node", "src/server.js"]
```

The image keeps the repository's layout (`backend/services/api` next to `frontend`) so the API finds the frontend at the same relative path as in development. It installs production dependencies only, runs as the unprivileged `node` user, and never contains a `.env` (`.dockerignore` excludes it); secrets are passed at run time. `docker-compose.yml` adds MongoDB 7 and Redis 7 with health checks, and starts the API only when both are healthy. Verified locally: the stack starts, the seed runs inside the container and `ledger:verify` reports a valid ledger.

16.4 Continuous integration

`.github/workflows/ci.yml` runs on every push to `main` and every pull request:

1. **backend**: starts MongoDB and Redis as service containers, installs dependencies with `npm ci` and runs the tests with coverage;
2. **frontend**: runs the jsdom tests;
3. **docker**: after both pass, builds the image.

The workflow has been written and its steps run locally; it has not yet run on GitHub at the time of writing.

16.5 Production checklist

- `NODE_ENV=production`; `EMAIL_TRANSPORT=smtp` with a working app password; fresh random JWT secrets.
- Serve over HTTPS (the refresh cookie is `Secure` in production) behind a reverse proxy.
- `ALLOWED_ORIGINS` restricted to real origins.
- MongoDB and Redis on a private network with authentication; ideally a MongoDB user that can insert but not modify `ledgerentries`.
- Back up MongoDB, and publish ledger head hashes regularly (Section 8.7).

CHAPTER 17

Demo, Security Summary and Future Work

17.1 Demo script

The full step-by-step script is in `docs/DEMO.md`. In outline (about 25 minutes, after `npm run seed --reset` and `npm run dev`):

1. **Landing page and login (2 min).** Live homepage counts. Sign in as `you+patient@gmail.com`: password, then the emailed 6-digit code. Reload the page: still signed in, through the HttpOnly refresh cookie.
2. **Patient, Arjun Mehta (5 min).** Book an appointment from the live slot grid. Open the hypertension record, download an attachment, *Verify integrity*, read the ledger history. Approve Dr Menon's access request. Request a refill. Show vitals charts. File a claim on an unpaid bill.
3. **Doctor, Dr Ananya Iyer (5 min).** Confirm the new booking (Arjun is notified). Complete a visit: a bill appears. Open Arjun's chart, write a record, upload a lab report, prescribe. Find Rohan Gupta by email: his chart is refused (403, no consent). Switch on *Emergency access* with a reason: the chart opens under a red banner, and Rohan and the admins are notified.
4. **Admin (4 min).** Live statistics. Approve Dr Kavya Reddy. Deactivate and reactivate a user. Register an IoT device (key shown once). Read the Access Control audit, including the break-glass entry. Ledger: *Verify ledger* reports a valid ledger.
5. **Tamper detection (2 min).** `npm run demo:tamper` edits Arjun's diagnosis directly in MongoDB. *Verify ledger* now reports tampering and names the record; Arjun's *Verify integrity* turns red. `npm run demo:tamper -- --restore`, verify again: valid.
6. **Live IoT (2 min).** `npm run demo:iot -- --interval 2` streams readings to the doctor's IoT Monitoring page; `--spike` sends abnormal values that raise alerts for the patient and doctor.
7. **Insurance agent, Priya Sharma (3 min).** Process the submitted claim: under review, approve with an amount, settle. Each step is a ledger entry and a notification to Arjun; his bill shows the covered amount.
8. **Failure states (1 min).** Stop the server: every screen shows "Server unreachable" with *Try again*. A new account shows real empty states.

17.2 Security summary

Table 17.1: Threats and controls

Threat	Control
Stolen password	Second factor (emailed OTP); lockout after 5 failures; rate limits
OTP guessing	6 random digits, 10-minute TTL, single use, per-email attempt limit, constant-time compare
Account enumeration	Same message for unknown email and wrong password
Stolen access token	15-minute life; revocable by `jti`; trust score drops with age
Stolen refresh token	HttpOnly + SameSite=Strict cookie; rotation makes reuse detectable; revocable
XSS	Strict CSP without inline scripts; every value escaped; tokens not in storage
CSRF	SameSite=Strict cookie limited to `/api/auth`; API uses Bearer headers
Privilege escalation	Role, approval and trust checked server-side on every route; unknown body fields rejected; admin not self-registrable
Doctor snooping	Patient consent with expiry required; break-glass audited and notified
Deactivated user keeps working	Session state re-checked per request (5-minute cache, invalidated on change)
Insider edits the database	Hash chain plus per-document payload hashes; `ledger:verify` and UI verification
Tampered file	SHA-256 at upload in the ledger; re-hashed on verify; hash header on download
Malicious upload	Type whitelist, 10 MB limit, served with `nosniff` and a fixed content type
Redis outage	Fail closed (503), never fail open
Leaked secrets	Only in environment; config validation; `.env` git-ignored and excluded from Docker images
Stolen device key	Only a hash stored; key bound to one patient; device can be deactivated

Action needed outside the code

The repository's history (commit `91a158e`) contains an old `backend/services/auth/.env` with a real Gmail address, Gmail app password and JWT secret. The current code does not use that file, but the app password should be revoked in the Google account it belongs to, because anyone with access to the history can read it.

17.3 Known limitations and future work

- **Ledger anchoring.** Publish chain head hashes externally and restrict the API's MongoDB user to insert-only on `ledgerentries` (Section 8.7). A move to Hyperledger Fabric would

replace `ledgerService` only.

- **Payments.** `POST /bills/:id/pay` records a payment; a real payment gateway is not integrated.
- **Performance and security testing.** A load test against the 200 ms / 2 s targets and an OWASP ZAP scan have not been run.
- **Staging deployment.** CI builds and tests; there is no staging environment to deploy to yet.
- **Real-time updates.** Dashboards poll (notifications every 20 s, IoT monitor every 5 s). WebSockets or server-sent events would push changes instantly.
- **Encryption at rest.** MongoDB's storage encryption or field-level encryption for diagnoses would protect against stolen disks or backups.
- **Time zones.** Slots are computed in the server's local time; a multi-site hospital would need per-hospital time zones.
- **Accessibility audit.** Forms are labelled and keyboard usable, but no formal WCAG audit has been done.

17.4 Verification record

What was actually verified for this version:

- Backend: 105 tests pass (Jest + Supertest, real MongoDB and Redis), about 93% line coverage; three consecutive runs were stable.
- Frontend: 20 tests pass (Jest + jsdom).
- In the browser: all four dashboards load every page without errors; the full OTP login (code read from the development log); booking, claim processing, break-glass and ledger verification through the UI; the "server unreachable" state with the server stopped; real empty states for a new account.
- Scripts: seed, IoT simulator, tamper demo with restore, ledger verifier.
- Docker: `docker compose up -build` produced a healthy stack that seeded and verified inside the container.
- Not verified: delivery through real Gmail SMTP (needs the owner's app password), the GitHub Actions run, load and ZAP testing.